
CHAIN-OF-THOUGHT SHOWS THE PATH TO A TREE: REALIZING BRANCHING COMPLEXITY

Debanjan Dutta
Indian Statistical Institute
Kolkata, India

Anish Chakrabarty
LTCI, Télécom Paris
Palaiseau, France

Swagatam Das[†]
Indian Statistical Institute
Kolkata, India

ABSTRACT

Chain of Thought (CoT) lifts the expressive ceiling of bounded-depth Transformers, with characterizations tying the number of CoT steps to circuit complexity classes. What remains largely missing are concrete instantiations with explicit, depth-bounded constructions, and the traversal procedures such characterizations presuppose. We close this gap for branching complexity. We give CoT realizations of depth-first search (DFS) and of Dijkstra algorithm, the latter subsuming breadth-first search, by unique hard-attention decoders of at most two layers, and use them as a shared computational substrate: reusing the DFS decoder yields the Strahler number of an n -vertex tree in $2n - 1$ steps with four layers, and reusing the Dijkstra decoder yields its width in $n - 1$ steps with three. Since computing the Strahler number of a binary tree given as a term is NC^1 -complete, and our constructions handle arbitrary n -ary trees without layer normalization or positional encodings, this is a non-trivial witness for the linear-step regime of the CoT hierarchy. Exploiting the classical bijection between ordered trees and Dyck paths, itself realized by our DFS construction, which emits the path as it traverses, we give independent constructions for both measures on the path representation.

1 Introduction

Chain-of-thought (CoT) prompting has turned autoregressive Transformers into systems that appear to execute procedures rather than merely predict tokens (Wei et al., 2022). Going beyond empirical curiosity, Merrill and Sabharwal (2024) show that CoT lets bounded-depth Transformers escape the complexity-class ceiling of a single forward pass, and Barceló et al. (2025) go further, giving an exact fixed-step characterization in terms of the Ehrenfeucht–Haussler (EH) rank of a function. Such *possibility* results still allow for explicit, minimal-depth CoT construction solving non-trivial problems exhibiting the traversal or recursion, which are only known to exist. Barceló et al. (2025) themselves note that their bound is achieved through exhaustive tree traversal, but they do not construct the traversal. Zhu et al. (2026) study whether continuous CoT can solve reachability, but only empirically. Our work begins with this fact in mind.

Branching complexity of a tree —measured classically by the Strahler number (Strahler, 1957) and, independently, by the width of a tree —is one of the oldest quantitative questions about hierarchical structure, with consequences ranging from the minimum number of registers needed to evaluate an arithmetic expression (Flajolet et al., 1979) to bounds on random tree shape (Addario-Berry et al., 2013) to the size of the largest antichain (Dilworth, 1950). Moreover, it turns out to be readily tied to the CoT lore. The Strahler number of binary trees and the EH rank on binary decision trees satisfy the same recursive update (Dahiya and Mahajan, 2021), and Ganardi and Lohrey (2026) have just shown that computing the Strahler number of a binary tree is NC^1 -complete —precisely the class Merrill and Sabharwal (2024) associate with a *linear* number of CoT steps. In simpler terms, we now know of a rank measure known to sit at the boundary of what constant (or, linear) CoT depth can express, and a general theorem stating that such measures should be expressible with the right traversal machinery. In this work, we introduce the machinery.

We give explicit constructions of depth-first search (DFS), breadth-first search (BFS, as a special case of Dijkstra’s algorithm), and Dijkstra, realized by Transformer decoders with two layers and two attention heads under CoT. The findings do not emerge as isolated expressivity results but are based on a *computational substrate* on which branching

[†]Corresponding author email: swagatam.das@isical.ac.in

complexity becomes CoT-accessible. For example, reusing the same DFS realization that produces the traversal order, we compute the Strahler number in $2n - 1$ steps for an n -vertex tree (Theorem 8), and reusing the Dijkstra realization, we compute the width of a tree in $n - 1$ steps (Theorem 13). Because computing the Strahler number of a general n -ary tree is a strict generalization of the binary, tree setting in which the NC^1 -completeness result was established, this is a non-trivial concrete witness for the Merrill and Sabharwal framework, obtained without appealing to auxiliary primitives such as layer normalization that prior CoT constructions have relied on.

The traversal machinery gives us a second, independent route to the same measures. The classical bijection between ordered trees and Dyck paths (Goldman and Sundquist, 1992) allows us to flatten a tree onto a one-dimensional lattice path while providing an algorithmic witness of the bijection. The fact that the number of full binary trees G_T with n internal vertices and Strahler number $\text{st}(G_T)$ turns out to be the same as the number of Dyck paths of length $2n$ whose height h satisfies $\lfloor \log_2(1 + h) \rfloor = \text{st}(G_T)$ (Flajolet et al., 1979; Addario-Berry et al., 2024) becomes crucial in context. The DFS realization of Theorem 5 is the map from tree to path, since a traverse step is exactly an up-step and a backtrack step is exactly a down-step. This raises a natural question that, to our knowledge, has not been asked in the CoT-expressivity literature: given two representations of the same combinatorial object related by an explicit, algorithmically realized bijection, do CoT realizations of a derived quantity (here, Strahler number and width) transfer across the bijection, or must they be constructed independently on each side? We answer this by proving identifiability results and associated constructions for both representations (Theorems 8, 11 for Strahler number; Theorems 13, 16 for width). The constructions, following Rizvi-Martel et al. (2024), involve bilinear maps. While we are interested in proving existence, an interesting open question emerges out of our inquiry: Is CoT realizability closed under bijective changes of representation, and hence, in general, under composition?

Below we summarize our main **contributions**.

- We give the first CoT realizations of general graph traversal (DFS and Dijkstra) by hard-attention Transformer decoders, each requiring at most two layers and two heads, and show that Dijkstra realization yields a step-count advantage over comparison-based implementations by replacing the search for a minimum-distance vertex with a single constant-time attention operation.
- We use these traversal realizations as a shared computational substrate to give explicit, small-depth CoT constructions for two independent notions of branching complexity, the Strahler number and width of a tree, providing a concrete, non-trivial instantiation of the CoT-rank framework after Barceló et al. (2025) on a measure already known to be NC^1 -complete in the binary case (Ganardi and Lohrey, 2026), generalized here to arbitrary ordered trees.
- Exploiting the classical bijection between trees and Dyck paths, realized algorithmically by our DFS construction, we give independent CoT constructions for both measures on the path representation, and show that the resulting constructions do not transfer readily across the bijection without change in mechanism or layer count.

Organization. section 2 introduces the necessary preliminaries on graph traversal and branching complexity. section 3 formalizes the CoT realization framework used throughout. section 4 contains the main results: traversal constructions (DFS and Dijkstra) (subsection 4.1) and realization of the Strahler number and tree width on both trees and Dyck paths along with the path-reconstruction algorithm (subsection 4.2). section 5 concludes with a comprehensive discussion. Full proofs are deferred to the Appendix.

Related Works. The expressivity of Transformers is well studied, from single-pass characterizations of encoders and decoders (Strobl et al., 2024) to step-indexed accounts of chain-of-thought decoders placing logarithmic and linear reasoning budgets in L and NC^1 respectively (Merrill and Sabharwal, 2024; Li et al., 2024). In parallel, graph-algorithmic constructions realize DFS, BFS, and Dijkstra via looped Transformers (Giannou et al., 2023) with graph-interacting heads (De Luca and Fountoulakis, 2024; Sanford et al., 2024), and empirical CoT studies address reachability without materializing traversal (Zhu et al., 2026); each lies outside the autoregressive step-counted CoT setting, leaving traversal yet to be explicitly realized as CoT steps. The concrete problems that inhabit the linear-step regime remain few. The Strahler–EH correspondence (Dahiya and Mahajan, 2021), together with the NC^1 membership

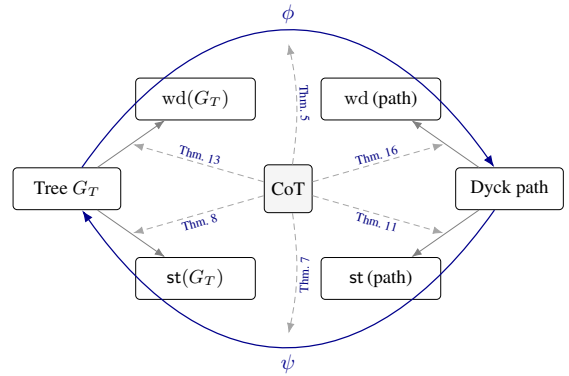


Figure 1: Overview of the paper’s constructions and their relationships. The unfolding, ϕ , maps a tree G_T to its Dyck path by recording the DFS traversal used in Theorem 5, and the reconstruction, ψ , is realized via Algorithm 1 (Theorem 7). Besides identifying st and wd across this bijection, we provide standalone CoT-realization pathways for each directly on both representations: G_T (Theorems 8 and 13, respectively) and paths (Theorems 11 and 16).

of Strahler number computation (Ganardi and Lohrey, 2026), motivates our focus on branching complexity. Appendix A presents a comprehensive discussion.

2 Branching Complexity and Graphs

2.1 Graphs and their traversal techniques

Let $G = (V, E, A)$ be a simple connected directed graph with vertex set $V = \{v_0, \dots, v_{n-1}\}$, edge set $E \subseteq V \times V$, and adjacency matrix $A : V \times V \rightarrow \mathbb{A}$ over an arbitrary domain $\mathbb{A}$, where $a_{ij} = A(v_i, v_j)$ denotes the weight of edge $e_{ij} = (v_i, v_j)$.

Dijkstra Algorithm. Restricting to $\mathbb{A} = \mathbb{R}_{>0}$ and fixing a source $s = v_p \in V$, the Dijkstra algorithm computes single-source shortest paths. The instantaneous description of the dynamic at step t is the tuple $\text{Dkst}^{(t)} = (\text{Dist}^{(t)}, \text{cur}^{(t)}, \text{vst}^{(t)})$, with $\text{Dist}^{(t)} \in (\mathbb{R}_{>0} \cup \{\infty\})^n$ being the tentative distance vector, $\text{cur}^{(t)} \in V$ the current vertex, and $\text{vst}^{(t)} : V \rightarrow \{0, 1\}$ the visited indicator. The system is initialized by $\text{cur}^{(0)} = v_p$, $\text{Dist}^{(0)}[p] = 0$, $\text{Dist}^{(0)}[i] = \infty$ for all $v_i \neq v_p$, and $\text{vst}^{(0)}(v_p) = 1$ with $\text{vst}^{(0)}(v_i) = 0$ otherwise. At each step, given $\text{cur}^{(t)} = v_i$, the transition relaxes all outgoing edges by setting $\text{Dist}^{(t+1)}[j] = \min(\text{Dist}^{(t)}[j], \text{Dist}^{(t)}[i] + a_{ij})$ for every $v_j \in V$, selects the next vertex $\text{cur}^{(t+1)} = v_{j'}$ where $j' = \arg\min_j \{\text{Dist}^{(t+1)}[j] \mid \text{vst}^{(t)}(v_j) = 0\}$, and marks $\text{vst}^{(t+1)}(v_{j'}) = 1$ while preserving all prior visits. At termination, $\text{Dist}^{(n-1)}[i] = \min_{P \in \mathcal{P}(s, v_i)} w(P)$ for every $v_i \in V$, where $\mathcal{P}(s, v_i)$ denotes the set of directed (s, v_i) -paths and $w(P) = \sum_{(v_k, v_l) \in P} a_{kl}$.

Observe that Dijkstra algorithm can be interpreted as a weighted generalization of the BFS traversal. In this context, we redefine $\mathbb{A} = \{1, \infty\}$ to indicate the presence or absence of an edge e_{ij} , where a value 1 denotes that the edge e_{ij} exists and ∞ implies that it does not.

Depth-First Traversal. For DFS traversal (Aho and Hopcroft, 1974) DFS, we choose $\mathbb{A} = \{0, 1\}$. Then, $\mathcal{N}(v_i) = \{v_j \mid a_{ij} = 1\}$ and let $n = |V|$. The instantaneous description of the dynamic DFS^(t) is denoted by the tuple $(\text{dfs}^{(t)}, \text{par}^{(t)}, \text{vst}^{(t)})$, such that $\text{dfs}^{(t)} : V \rightarrow V$, $\text{par}^{(t)} : V \rightarrow V \cup \{\phi\}$ and $\text{vst}^{(t)} : V \rightarrow \{0, 1\}$ returns an spanning tree T from any basic diagraph (Gessel and Wang, 1979) G in $O(|V| + |E|)$ moves. At $t = 0$, given a starting vertex $s = v_p \in V$, $\text{vst}^{(0)}(s) = 1$ and $\text{vst}^{(0)}(v_i) = 0$ for all $v_i \in V \setminus \{s\}$ and $\text{par}^{(0)}(v_i) = \phi$ for all v_i . Then the t^{th} dynamic is given by

$$\text{dfs}^{(t)}(s) = \begin{cases} v_j & \text{if } v_j \in \mathcal{N}(\text{dfs}^{(t-1)}(s)) \text{ with } \text{vst}^{(t-1)}(v_j) = 0, \\ \text{par}^{(t-1)}(\text{dfs}^{(t-1)}(s)), & \text{otherwise.} \end{cases} \quad (1)$$

In the former case, *traverse*, $\text{par}^{(t)}(v_j) = \text{dfs}^{(t-1)}(s)$ and $\text{vst}^{(t)}(v_j) = 1$; all remaining entries of $\text{par}^{(t)}$ and $\text{vst}^{(t)}$ are inherited from step $t - 1$. In the latter case, *backtrack*, the mappings remain unchanged. The algorithm terminates after backtracking from s .

Throughout the remainder, we write $n = |V|$ unless otherwise stated, with n also denoting the length of a lattice path where relevant. Furthermore, whenever the edge set A is omitted in the definition of a graph, we refer to its unweighted variant.

2.2 Trees, Paths, and Branching Complexity

If in a graph $G_T = (V, E)$ there is exactly one path between any two distinct vertices $v_i, v_j \in V$, then G_T is a tree, and $|E| = |V| - 1$. For a finite rooted tree G_T (Diestel, 2017), let $\text{depth}(v)$ denote the number of edges on the unique path from the root r to vertex v . The height of G_T is defined as $\text{ht}(G_T) := \max_{v \in V} \text{depth}(v)$, and width as

$$\text{wd}(G_T) := \max_{k \geq 0} |\{v \in V : \text{depth}(v) = k\}|.$$

Observation 1 (Determining wd while performing BFS). To algorithmically determine $\text{wd}(G_T)$, the width of tree G_T , we employ BFS as a special version of the Dijkstra algorithm. Let v_j and $v_{j'}$ be the vertices such that $\text{vst}^{(t)}(v_j) = \text{vst}^{(t)}(v_{j'}) = 0$; $\text{vst}^{(t+1)}(v_j) = 1, \text{vst}^{(t+1)}(v_{j'}) = 0$; and $\text{vst}^{(t+2)}(v_j) = \text{vst}^{(t+2)}(v_{j'}) = 1$. Note that, $\text{Dist}^{(t+2)}[j'] - \text{Dist}^{(t+2)}[j] \in \{0, 1\}$. Clearly, when $\text{Dist}^{(t+2)}[j'] = \text{Dist}^{(t+1)}[j]$, $\text{depth}(v') = \text{depth}(v)$; and $\text{depth}(v') = \text{depth}(v) + 1$, otherwise. To capture the width during $\text{Dkst}^{(t)}$, we augment it with scalars $\text{twd}^{(t)}$ and $\text{mwd}^{(t)} \in \mathbb{Z}$ so that the former stores $|\{v_i : \text{vst}^{(t)}(v_i) = 1 \text{ and } \nexists j (\text{Dist}^{(t)}[j] \neq \infty \wedge \text{Dist}^{(t)}[j] > \text{Dist}^{(t)}[i])\}|$ while the latter stores $\max_{t' < t} \text{twd}^{(t')}$. Unless G_T is a null graph, $\text{twd}^{(t)}$ and $\text{mwd}^{(t)}$ for all t must be at least 1. Then initialized by $\text{twd}^{(0)} = \text{mwd}^{(0)} = 1$ indicating the root is at depth 1, $\text{mwd}^{(n-1)} = \text{wd}(G_T)$.

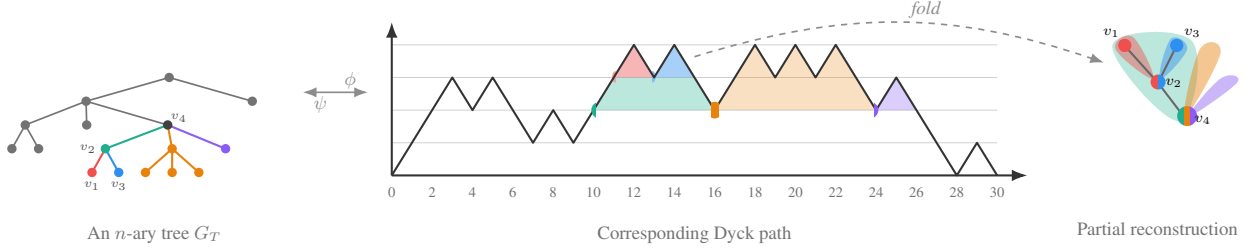


Figure 2: Following the DFS dynamic, the map ϕ on the tree G_T (left) generates the Dyck path (middle). We define the reconstruction ψ of the tree from the Dyck word (or path) via [Algorithm 1](#), with a demonstrative (partial) reconstruction illustrated on the right. The edge pairs $(11, 3)$, $(12, 4)$ and $(12, 4)$, $(13, 3)$ are folded into a single tree edge (v_1, v_2) . The highlighted region on the left is formed by folding the region between $(10, 2)$ and $(16, 2)$.

Although this is a very natural notion of branching complexity, the [Strahler](#) number st , on the other hand, is calculated recursively. For leaf node v , $\text{st}(v) = 0$ and a node with subtrees $\tau_1, \dots, \tau_k$, let $M = \max_i \text{st}(\tau_i)$. Then,

$$\text{st}(G_T) = \begin{cases} M + 1 & \text{if } c = |\{i : \text{st}(\tau_i) = M\}| \geq 2 \\ M & \text{otherwise.} \end{cases}$$

For binary trees, this reduces to $\text{st}(G_T) = \max(\min(\text{st}(\tau_1), \text{st}(\tau_2)) + 1, \max(\text{st}(\tau_1), \text{st}(\tau_2)))$, which, interestingly, coincides with the expression of the Ehrenfeucht-Haussler rank for binary decision trees.

Observation 2 (Determining M and c whiler performing DFS). We reuse the dynamic formulation (1), which is evaluated in a bottom-up manner. Specifically, when the dynamic process is in a *traverse* step, we assign $t_{\text{dfs}(t)(s)}$, the tree rooted at the vertex $\text{dfs}^{(t)}(s)$, the value 0. Note that in this situation, $\tau_{\text{dfs}(t)(s)}$ consists of a single vertex. During a *backtrack* step, let $v = \text{dfs}^{(t)}(s)$ and $u = \text{dfs}^{(t+1)}(s)$. At this point, we record $\text{st}(\tau_u)$ for the tree rooted at the vertex u that has been explored up to step t . We let $M_u^{(t)}$ and $c_u^{(t)}$ denote, respectively, the maximum Strahler number observed among the already explored children of u other than v ¹, and the number of children attaining this maximum value. Analogously, $M_v^{(t)}$ and $c_v^{(t)}$ denote the maximum Strahler number observed among the children of v and the corresponding multiplicity of this maximum. Then $\text{st}(\tau_v) = M_v^{(t)} + \mathbb{1}[c_v^{(t)} \geq 2]$. Let $D = \text{st}(v) - M_u^{(t)}$.

$$D \begin{cases} < 0 & M_v^{(t+1)} = M_u^{(t)}, c_v^{(t+1)} = c_u^{(t)} \\ = 0 & \begin{cases} M_v^{(t+1)} &= M_u^{(t)} + \mathbb{1}[c_u^{(t)} \geq 1] \\ c_v^{(t+1)} &= \mathbb{1}[c_u^{(t)} + 1 < 2] \end{cases} \\ > 0 & M_v^{(t+1)} = \text{st}(\tau_v), c_v^{(t+1)} = 1, \end{cases} \quad (2)$$

which reduces to

$$\begin{aligned} M_v^{(t+1)} &= \max(\text{st}(\tau_v), M_u^{(t)}) + \mathbb{1}[D = 0] \mathbb{1}[c_u^{(t)} > 0] \\ c_v^{(t+1)} &= \mathbb{1}[D < 0] c_u^{(t)} + \mathbb{1}[D = 0] \mathbb{1}[c_u^{(t)} = 0] + \mathbb{1}[D > 0] 1. \end{aligned}$$

A lattice path of length n with step set $\mathcal{S} \subseteq \mathbb{Z}$ is a sequence $(0, s_0), (1, s_1), \dots, (n, s_n)$ with $s_0 = 0$ and $s_i - s_{i-1} \in \mathcal{S}$ for all $i \geq 1$; it is an *excursion* [Flajolet and Sedgewick \(2009\)](#) if $s_i \geq 0$ throughout and additionally $s_n = 0$. An excursion with step-set $\mathcal{S} = \{+1, -1\}$ is famously known as a Dyck path. By writing U for $+1$ and D for -1 , Dyck paths are presented as Dyck words over $\{U, D\}$. The bijection between trees and these paths is well known [Goldman and Sundquist \(1992\)](#). Given G_T , we run a DFS starting from root r and record a U each time the first case of the dynamic dfs occurs, and a D, otherwise. Since every one of the $n - 1$ edges is traversed once in each direction, the resulting word $\phi(G_T)$ has length $2(n - 1)$. Going the other way, given an excursion w of length $2m$, we build the tree $\psi(w)$: a U creates a new node attached as the next ordered child of the current node and moves the pointer down to it, while a D returns to the parent (see [Figure 2](#)). Therefore,

Lemma 1. *There exists maps ϕ and ψ , mutually inverse bijections between the set $\mathcal{T}_n$ of all finite ordered trees and the set $\mathcal{D}_{n-1}$ of Dyck words of length $2(n - 1)$.*

The bijection ϕ reduces the combinatorial structure of G_T to the geometry of $\phi(G_T)$. We make this precise for both ht and wd. Throughout, for $G_T \in \mathcal{T}_n$ we let $(s_0, s_1, \dots, s_{2(n-1)})$ denote the ordinate sequence of the lattice path induced

¹or one more than it, in a boundary case when $D = 0$ is observed a priori.

in $\phi(G_T)$: concretely, $s_0 = 0$ and $s_i - s_{i-1} = +1$ or -1 according to the i -th letter of $\phi(G_T)$ being U or D, so that $s_i = \sum_{j=1}^i q_j$ where $q_j \in \{+1, -1\}$ is the signed value of the j -th step.

Proposition 2. For every $G_T \in \mathcal{T}_n$, $\text{ht}(G_T) = \max_{0 \leq i \leq 2(n-1)} s_i$.

Proposition 3. For every $G_T \in \mathcal{T}_n$ with $n \geq 2$, $\text{wd}(G_T) = \max_{0 < l \leq \text{ht}(G_T)} |\{i : s_i = l, s_i - s_{i-1} = 1\}|$.

Remark 3.1 (Regularity and Relation between the Metric Measure Spaces). The existence and identifiability results above become even more important given the scarcity of similar evidence in a geometric setting, let alone their algorithmic realizability. Plane trees with n edges induce a deterministic bijection with Dyck paths of length $2n$ via the contour walk (Viennot, 2002) and with Łukasiewicz paths via the depth-first walk (Gall, 2005). However, the regularity of the same is not trivially observed. One can only ensure at the continuum for a continuous excursion $g : [0, 1] \rightarrow \mathbb{R}_+$ into the metric measure space of trees $\mathcal{T}_g^2$ the 2-Lipschitz-regularity of the forward map (in terms of the Gromov-Hausdorff distance) (Evans (2008), Gall (2005), Lemma 2.2.3). Then, the inverse embedding (tree $\rightarrow$ path) also fails to be Lipschitz, and thus admits no isometric isomorphism at the continuum level. A deterministic isomorphism of lattices (and hence a graph isometry of the associated metric spaces) can only be realized under triangulation (both endowed with the Tamari partial order) (Bernardi and Bonichon, 2009). While we leave checking the regularity of (ψ, ϕ) as future work, it can be noted as a first attempt at scrutinizing their CoT realizability.

To construct $\psi(w)$ from w , we introduce Algorithm 1 that returns the adjacency matrix of the tree on receiving a Dyck path as input (Appendix B provides the proof of correctness).

Algorithm 1. Reconstruction: Path $\rightarrow$ Tree (ψ)

Input: A Dyck word w corresponding to lattice path $\{(t, s_t)\}_{t=0}^{2(m-1)}$.

- 1 Let A_w be the adjacency matrix of $\psi(w)$ initialized to 0.
- 2 $nl \leftarrow 0, i \leftarrow 0$.
- 3 **for** $c \in w$
- 4 **if** c is U $\triangleright$ i.e. $s_{t+1} - s_t = 1$.
- 5 $nl \leftarrow nl + 1$.
- 6 $A_w[i, nl] \leftarrow 1, A_w[nl, i] \leftarrow 1$.
- 7 $i \leftarrow nl$.
- 8 **else**
- 9 Search $j < i$ s.t. $A_w[j, i] = 1$.
- 10 $i \leftarrow j$.
- 11 **return** A_w .

3 Transformer Decoder and CoT

The CoT steps formalize, following Merrill and Sabharwal (2024); Li et al. (2024), the autoregressive nature of the Transformer decoder. We first describe the *unique hard*-attention layer and then the full decoder architecture. Our framework largely follows Barceló et al. (2025), with two necessary modifications that we identify in our exposition.

Unique Hard-Attention. Given an input sequence $(x_0, \dots, x_{m-1}) \in (\mathbb{R}^d)^m$, let $Q, K \in \mathbb{R}^{d' \times d}$ denote the query and key projection matrices, respectively. The attention score between the position i and the query position $m-1$ is defined as $a_{i,m-1} = \langle Kx_i, Qx_{m-1} \rangle$. With respect to the m -th query Qx_{m-1} , the scalar $a_{i,m-1}$ quantifies the affinity of the key Kx_i . Let $j_0 < j_1 < \dots < j_p \in \{0, \dots, m-1\}$ denote the indices, arranged in ascending order, that jointly attain the maximum attention score. Unique hard attention selects the smallest such index j_0 . Departing from Barceló et al. (2025), we retain the standard value matrix³ $V \in \mathbb{R}^{d'' \times d}$, so that the attention produces Vx_{j_0} .

Multi-Head Attention. A multi-head extension uses H attention heads, where head h ($h \in [H]$) is parameterized by (Q_h, K_h, V_h) . Let $j_0^{(h)}$ be the index selected by head h ; the combined output is

$$\alpha'' = W_O \cdot \left[\left(V_1 x_{j_0^{(1)}} \right)^\top \parallel \dots \parallel \left(V_H x_{j_0^{(H)}} \right)^\top \right]^\top,$$

²endowed with the pseudometric $d_g(s, t) := g(s) + g(t) - 2 \min_{[s \wedge t, s \vee t]} g$, after quotienting by $\{d_g = 0\}$ and measure $\mu_g = (p_g)_* \mu$, where μ is the Lebesgue measure.

³The symbol V is overloaded to denote both the vertex set of a graph and the value projection in attention; the intended meaning will be clear from context.

where $\parallel$ denotes vector concatenation, W_O is the output projection matrix, and $\alpha'' \in \mathbb{R}^d$.

Single-Layer Decoder. A position-wise feed-forward network is applied to the residual $\beta + x_{m-1}$ via two successive linear projections interleaved with a ReLU nonlinearity, where $\text{ReLU}(r) := \max(0, r)$ for $r \in \mathbb{R}$, applied coordinate-wise for $r \in \mathbb{R}^d$. The single-layer decoder thus computes

$$L(x_0, \dots, x_{m-1}) = W_2 \text{ReLU}(W_1(\beta + x_{m-1})),$$

for weight matrices $W_1 \in \mathbb{R}^{d''' \times d}$ and $W_2 \in \mathbb{R}^{d \times d'''}$. In our constructions, we will often use α' to denote Vx . Each input vector x_i is constructed from a word embedding $\text{WE}(\sigma_i)$ and a positional encoding $\text{pos}(i)$ via a composition $x_i = g(\text{WE}(\sigma_i), \text{pos}(i))$. Unlike Barceló et al. (2025), we adopt the simplification $g(\text{WE}(\sigma_i), \text{pos}(i)) = \text{WE}(\sigma_i) =: x_i$. A single-layer Transformer decoder, upon receiving input $(x_0, \dots, x_{m-1}) \in (\mathbb{R}^d)^m$ and an initial eos token $y_0 \in \mathbb{R}^d$, generates output sequence $\{y_t \in \mathbb{R}^d\}_{t=1}^\infty$ via the recurrence

$$y_t = L(x_0, \dots, x_{m-1}, y_0, \dots, y_{t-1}), \quad t \geq 1.$$

That is, at each step t , the decoder appends its most recent output y_{t-1} to the running context and applies L to the resulting extended sequence.

Multi-Layer Extension. A depth- ℓ Transformer decoder is obtained by composing ℓ such single-layer decoders $L^{(1)}, \dots, L^{(\ell)}$, each with its own independent parameter set $\{Q_h^{(k)}, K_h^{(k)}, V_h^{(k)}, W_O^{(k)}, W_1^{(k)}, W_2^{(k)}\}_{h=1}^{H_k}$ for layer $k \in [\ell]$. Given the input context $(x_0, \dots, x_{m-1})$, the layers process the sequence in depth-first order: at generation step t , the intermediate representations are computed as

$$z_m^{(0)} := x_{m-1}, z_m^{(k)} := L^{(k)}(z_1^{(k-1)}, \dots, z_m^{(k-1)}),$$

for $k \in [\ell]$ and the final output at step t is $y_t := z_{m_t}^{(\ell)}$, where $m_t = m + t$ denotes the current sequence length. The newly produced token y_t is appended to the context, after which the full ℓ -layer computation is repeated for step $t + 1$.

Definition 3.1 (CoT Realization). Let $\mathcal{A}$ be an algorithm that, on input, I say, produces a sequence of states $\{s_t\}_{t=1}^T$, where T denotes the number of iterations until termination. Let $\pi : \mathbb{R}^d \rightarrow \mathbb{R}^{d_s}$ denote the projection onto the designated output coordinates. We say that a decoder *realizes* $\mathcal{A}$ via CoT if there exists a parameter assignment such that, for every valid input and every $t \in [T]$, $\pi(y_t) = \text{enc}(s_t)$, where enc is a fixed encoding of algorithmic states into $\mathbb{R}^{d_s}$. The generation runs for exactly T steps; the step count T is determined a priori, and hence termination follows immediately.

4 Main Results

This section is organized as follows. First, we present the construction of the graph traversal techniques (see section 4.1). Section 4.2 then introduces the theorems necessary to establish the main results, as summarized in Figure 1. The bilinear maps employed in our constructions, following Rizvi-Martel et al. (2024), are in general not unique, as can be seen by comparing Theorem 8 and 11. However, substituting these bilinear maps with FFN blocks from a Transformer layer would, in several cases where merging independent operations is infeasible, alter the required number of layers.

Bilinear maps have been employed in the study of Transformers in both theoretical analyses (Rizvi-Martel et al., 2024) and empirical investigations, particularly within the contexts of multi-modal and multi-view learning (Li et al., 2017; Gao et al., 2016). In our work, we adopt analogous bilinear projection mechanisms as fundamental components of our constructions. We apply linear, affine, and bilinear maps to independent resultant blocks using a single transformation. In Appendix C, we show that all of these can be written as a single bilinear map. Also, any scalar–vector operation is applied elementwise to the vector.

4.1 Results on Graph Traversal

Theorem 5. A two-layer, two-head Transformer decoder can simulate the depth-first traversal DFS on any simple directed graph $G = (V, E, A)$ in $O(|V| + |E|)$ chain-of-thought steps.

Proof Sketch. The construction realizes the dynamic $\text{dfs}^{(t)}$ as in (1) and proceeds in two layers. The first layer uses a trivial attention mechanism and determines, via its feed-forward block, whether the current vertex has an unvisited neighbor — distinguishing a *traverse* from a *backtrack*. The second layer executes the chosen operation through two mutually exclusive attention heads, one resolving the next unvisited neighbor and the other retrieving the parent vertex (see Appendix D). $\square$

When the underlying graph is a tree G_T , the number of required CoT steps is $2(n - 1)$. The DFS realization plays a central role in two distinct respects: *i*. The mapping ϕ coincides exactly with the execution of the procedure DFS on the tree G_T . To obtain the associated Dyck path (or Dyck word), it suffices to extend the token embedding by one additional component that records $+1$ (or U) and -1 (or D) exclusively, in accordance with the *traverse* and *backtrack* operations, in the final layer. *ii*. The same DFS realization is also reused in the implementation of (2), thereby yielding the Strahler number of a tree as stated in [Theorem 8](#).

Theorem 6. A two-layer, single-head Transformer decoder can simulate the Dijkstra algorithm Dkst on any simple connected directed graph $G = (V, E, A)$ with $A = \mathbb{R}_{>0}$ in exactly $|V| - 1$ chain-of-thought steps.

Proof Sketch. The proof proceeds by implementing the dynamics of $\text{Dkst}^{(t)}$ as described in [subsection 2.1](#). In the first layer, the attention collects the edge weights by identifying the current vertex from input tokens. Since $A \in \mathbb{R}_{>0}$, its FFN block updates the distance vector Dist by applying the relaxation operation to all neighboring vertices. The second layer then uses its attention mechanism to select the unvisited vertex with the smallest Dist , updates its representation, and marks the selected vertex as visited (see [Appendix E](#)). $\square$

The classical implementation of Dijkstra algorithm requires $O(n^2)$ time, whereas efficient variant runs in $O(|E| + n \log n)$. In contrast, our implementation requires at most $n - 1$ CoT steps. This yields a substantial computational advantage: the Transformer avoids explicitly searching for the minimum-distance unvisited vertex by using constant-time attention. We use this theorem, along with [Observation 1](#), to compute wd in [Theorem 13](#).

4.2 CoT Realization of Complexity Measures

After proving [Algorithm 1](#) in [Theorem 7](#) (see [Appendix F](#)), [Theorem 8](#) and [11](#) present the proofs for computing the Strahler number treating tree and Dyck path as inputs, respectively, under CoT ([Appendix G](#)). [Theorem 13](#) and [16](#) provide the proofs for computing the measure wd ([Appendix H](#)).

Theorem 7. A single-layer, two-head Transformer decoder can simulate [Algorithm 1](#) in the $2(m - 1)$ CoT-steps⁴.

Proof Sketch. Each step executes one iteration of the loop on line 3, reading the symbol w_t and updating (i, nl, A_w) in place. A single attention head reads w_t by matching the cursor pos against the input tokens, returning its signed increment $q_t \in \{+1, -1\}$; a second head, querying on the current vertex, retrieves from that vertex’s creation token the parent needed on a D step (line 9). The feed-forward block then branches on the sign of q_t entirely within a single ReLU, via $\text{ReLU}(v + q_t - 1) = \mathbb{1}[q_t = +1]v$: on a U it increments nl , writes the symmetric edge $\{i, nl\}$ as the one quadratic feature of W_1 , and advances i to the new vertex (lines 5–6); on a D it moves i to the retrieved parent. The cursor advances each step, and the chain halts when the path exhausts. Induction on t shows $\pi_A(y_t)$ equals the adjacency matrix of $\psi(w)$ after reading $w_0 \cdots w_{t-1}$. $\square$

Theorem 8. A four-layer, two-head Transformer decoder can compute $\text{st}(G_T)$, the Strahler number of a tree G_T during the simulation of DFS in exactly $2n - 1$ CoT steps, where n denotes the number of vertices in G_T .

Proof Sketch. The decoder of [Theorem 5](#) traverses G_T . We extend its embedding alone, giving each token a pair (π_M, π_c) : the running maximum of the Strahler numbers of the current vertex’s finished children, and the count attaining it. A timestamp π_{flg_d} turns the backtrack head’s leftmost selection into a rightmost one, so it resolves the parent as last left and reads the accumulator it then carried. The feed-forward blocks of two further layers fold the finished child into this accumulator. The comparison against the parent’s maximum is decided through indicators of $D = \text{st}(v) - M_u$ and of the count, laid down as differences of rectified blocks. A carry lifts the maximum when the count saturates, keeping the fold bilinear. The accumulator is never finalized in place; the final *backtrack* folds the source under the same invariant, giving $\pi_M(y_{2n-1}) = \text{st}(G_T)$ in $2n - 1$ steps. $\square$

Theorem 11. A four-layer, single-head Transformer decoder can compute $\text{st}(\psi(w))$, the Strahler number of a tree corresponding to the Dyck word w in exactly n CoT steps, where $n = |w|$.

The proof of the above theorem builds on the same techniques as before, in particular the implementation of (2) from [Theorem 8](#). For this reason, it is only outlined in [Appendix G](#), where we provide the additional arguments needed to handle paths as input. Note that while [Theorem 8](#) directly computes the st number of the tree G_T in its final CoT step, the above theorem provides the values M and c in the designated positions as specified by $\pi_M(y_n)$ and $\pi_c(y_n)$ so that $\text{st}(\psi(w)) = \pi_M(y_n) + \mathbb{1}[\pi_c(y_n) \geq 2]$.

⁴In this derivation alone, we write the length of the Dyck path for a tree of m nodes as $2(m - 1)$, to make the distinction explicit; elsewhere we retain n , as mentioned earlier, for the number of nodes or the Dyck path length.

Theorem 13. A three-layer single-head Transformer decoder can find the width of a tree $\text{wd}(G_T)$ during the simulation of Dkst in exactly $n - 1$ steps, where n denotes the number of vertices in G_T .

Proof Sketch. We augment the embeddings of the tokens of Theorem 6 by appending three scalar coordinates $(\pi_{dif}, \pi_{twd}, \pi_{mwd})$, initialized to $(0, 1, 1)$ at the root, tracking a BFS level-change indicator, the running node count at the current level, and the global maximum width. Layers 1-2 replicate the Dijkstra construction, with the sole modification that $W_2^{(2)}$ additionally writes $\pi_{dif}(y_t^{(2)}) \leftarrow \sum_i \pi_{buf}(y_t^{(1)})[i] - \pi_{crd}(y_t)$, the difference between the BFS depths of the current and preceding decoded nodes. Since G_T carries weights in $\{1, \infty\}$ and nodes are decoded in non-decreasing BFS-depth order, Observation 1 guarantees $\pi_{dif} \in [0, 1]$, with $\pi_{dif} = 1$ signalling a transition to a strictly deeper level. Layer 3 employs a trivial attention sub-layer followed by an FFN that, when $\pi_{dif} = 0$, increments π_{twd} and updates π_{mwd} with $\max(\pi_{mwd}, \pi_{twd})$, and when $\pi_{dif} = 1$, commits the current count to π_{mwd} and resets π_{twd} to 1. After $n - 1$ decoding steps, each BFS level’s count has been committed to π_{mwd} , and thus $\pi_{mwd}(y_{n-1}) = \text{wd}(G_T)$. $\square$

Theorem 16. A two-layer single-head Transformer decoder can find $\text{wd}(\psi(w))$ for a Dyck word w in exactly n CoT steps, where $|w| = n$.

Proof Sketch. The embedding dimension $d = \frac{3n}{2} + 4$ is partitioned into a one-hot position block, a step scalar q_t , a cumulative height scalar ht , a count block $p \in \mathbb{R}^{\frac{n}{2}+1}$ where p_i tallies up-steps reaching height i , and a width scalar wd . In the first Layer, attention retrieves the current step q_t via a shift matrix, and the FFN applies a discrete pulse function (Lemma 14) to increment exactly the p_i entry indexed by the current height ht . In the final layer, the FFN performs a global max-aggregation (Lemma 15) updating wd to $\max_i p_i$ while zeroing the step scalar. Correctness follows since each p_i counts nodes at depth i in $\psi(w)$, so the final wd equals $\text{wd}(\psi(w))$ as claimed in Proposition 3. $\square$

A natural question arises regarding the optimal use of layers: given a tree G_T , is there an advantage to directly computing its width $\text{wd}(G_T)$ using a three-layer Transformer (as in Theorem 13), rather than breaking the process down? For instance, an alternative composite setup might combine a two-layer, two-head Transformer to implement the map ϕ (in Theorem 5) with a two-layer, single-head Transformer to compute width on the corresponding Dyck path (in Theorem 16). If CoT is closed under composition and layer counts add linearly, the first approach is strictly more economical in layer count. Moreover, directly employing the three-layer Transformer offers another benefit: it reduces the required CoT steps significantly compared to the composite method.

Remark 3.2 (Closure under Composition). Given the fact that CoT-Transformers fail to solve Compositional Reasoning Questions (CRQ) unless CoT tokens are allowed to grow accommodating compositional depth (Yehudai et al., 2025), it is clear that the composition does not replicate itself. However, the same constructive realizability witness result for composition over tree-structured tasks hints towards a linear complexity class. Notably, our construction preserves linear token growth. On the other hand, since ensuring CoT-learnability essentially boils down to the finiteness of the VC-dimension of the base classes $\mathcal{F}_i$ (Joshi et al., 2025), it would be sufficient for them to satisfy a VC upper bound showing linear aggregation in the spirit of Alon et al. (2021), Proposition 2: $\text{VC}(G(\mathcal{F}_1, \dots, \mathcal{F}_k)) \lesssim \text{VC}(G) + \sum_{i=1}^k \text{VC}(\mathcal{F}_i)$, to prove closure under composition along those lines, where G specifies composition, $k > 0$.

5 Conclusion

To our knowledge, the CoT realizations developed here are the first of their kind in the literature. This immediately invites the question of optimality: identifying constructions that minimize embedding dimension, layer depth, and number of attention heads constitutes a natural and important direction for future work. A second open question concerns the closure properties of CoT under the tree–path bijection. The CoT realizations of the Strahler number for path and tree inputs exhibit remarkably little structural synergy — beyond their shared adherence to (2) — despite the bijective correspondence between the two input representations via ϕ and ψ . A further observation concerns the number of heads required in Theorem 8 and Theorem 11. Intuitively, a single head can scarcely distinguish between a *traverse* and a *backtrack* along the same edge, whereas the linear progression of U and D in a Dyck path makes this distinction straightforward, so one head suffices. This also explains the two heads in Theorem 7: reconstructing the tree from the path re-encodes each D with its corresponding U, folding the information of both the *traverse* and *backtrack* steps back onto a single edge — and it is this folding that necessitates the second head.

Limitations

The constructions presented herein are not necessarily unique, and we acknowledge that alternative formulations satisfying the same theoretical requirements may well exist. As one illustration, [Theorem 8](#) introduces bilinear gating at the final layer to suppress redundant operations on the M and c blocks during a *traverse* step; [Lemma 10](#), however, demonstrates that bilinearity can be entirely circumvented to achieve the same effect in the context of [Theorem 11](#). As one limitation, several of our constructions invoke the bilinear operation. Following [Rizvi-Martel et al. \(2024\)](#), however, this usage stays within the established conventions of this line of work rather than constituting a departure from them. In this regard, unlike [Merrill and Sabharwal \(2024\)](#); [Qiu et al. \(2025\)](#), our constructions forgo supplementary computational primitives such as layer normalization. Rather than a shortcoming, this economy proved crucial for constructing CoT Transformers that reach NC^1 , showing that such augmentations are not a necessary architectural ingredient for CoT constructions. Additionally, we note that [Theorem 11](#) concludes with the final values M and c , the closing operation $M + \mathbb{1}[c \geq 2]$ that fixes the Strahler number sits marginally outside the CoT iteration. Furthermore, there is also a classical correspondence with plane trees and Łukasiewicz paths. We have not studied CoT realizations for these representations, which may yield additional architectural insights in the broader theory of computation.

References

- Louigi Addario-Berry, Marie Albenque, Serte Donderwinkel, and Robin Khanfir. 2024. [Refined Horton-Strahler numbers I: a discrete bijection](#). *Preprint*, arXiv:2406.03025.
- Louigi Addario-Berry, Luc Devroye, and Svante Janson. 2013. [Sub-Gaussian tail bounds for the width and height of conditioned Galton–Watson trees](#). *The Annals of Probability*, 41(2):1072 – 1087.
- Alfred V Aho and John E Hopcroft. 1974. *The Design and Analysis of Computer Algorithms*. Pearson Education India.
- Noga Alon, Alon Gonen, Elad Hazan, and Shay Moran. 2021. [Boosting simple learners](#). In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2021, page 481–489, New York, NY, USA. Association for Computing Machinery.
- Pablo Barceló, Alexander Kozachinskiy, Anthony Widjaja Lin, and Vladimir Podolskii. 2024. [Logical Languages Accepted by Transformer Encoders with Hard Attention](#). In *The Twelfth International Conference on Learning Representations*.
- Pablo Barceló, Alexander Kozachinskiy, and Tomasz Steifer. 2025. [Ehrenfeucht-Haussler Rank and Chain of Thought](#). In *Forty-second International Conference on Machine Learning*.
- Olivier Bernardi and Nicolas Bonichon. 2009. [Intervals in Catalan lattices and realizers of triangulations](#). *Journal of Combinatorial Theory, Series A*, 116(1):55–75.
- David Chiang, Peter Cholak, and Anand Pillay. 2023. Tighter Bounds on the Expressivity of Transformer Encoders. In *Proceedings of the 40th International Conference on Machine Learning*, ICML’23. JMLR.org.
- Yogesh Dahiya and Meena Mahajan. 2021. [On \(Simple\) Decision Tree Rank](#). In *41st IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2021)*, volume 213 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 15:1–15:16, Dagstuhl, Germany. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- Artur Back De Luca and Kimon Fountoulakis. 2024. Simulation of Graph Algorithms with Looped Transformers. In *International Conference on Machine Learning*, pages 2319–2363. PMLR.
- Reinhard Diestel. 2017. [Graph Theory](#), 5 edition, volume 173 of *Graduate Texts in Mathematics*. Springer, Heidelberg.
- E. W. Dijkstra. 1959. [A Note on Two Problems in Connexion with Graphs](#), 1 edition, page 287–290. Association for Computing Machinery, New York, NY, USA.
- R. P. Dilworth. 1950. [A Decomposition Theorem for Partially Ordered Sets](#). *Annals of Mathematics*, 51(1):161–166.
- Debanjan Dutta, Anish Chakrabarty, Faizanuddin Ansari, and Swagatam Das. 2025. On the Existence of Universal Simulators of Attention. *arXiv preprint arXiv:2506.18739*.
- Steven Neil Evans. 2008. *Probability and Real Trees: École D’Été de Probabilités de Saint-Flour XXXV-2005*. Springer.
- P. Flajolet, J.C. Raoult, and J. Vuillemin. 1979. [The Number of Registers Required for Evaluating Arithmetic Expressions](#). *Theoretical Computer Science*, 9(1):99–125.
- Philippe Flajolet and Robert Sedgewick. 2009. [Analytic Combinatorics](#). Cambridge University Press, Cambridge.
- Jean-François Le Gall. 2005. [Random trees and applications](#). *Probability Surveys*, 2(none):245 – 311.

- Moses Ganardi and Markus Lohrey. 2026. [On the Complexity of Computing Strahler Numbers](#). In *43rd International Symposium on Theoretical Aspects of Computer Science, STACS 2026, Grenoble, France, March 9-13, 2026*, LIPIcs, pages 41:1–41:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik.
- Yang Gao, Oscar Beijbom, Ning Zhang, and Trevor Darrell. 2016. Compact Bilinear Pooling. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Ira Gessel and Da-Lun Wang. 1979. [Depth-first Search as a Combinatorial Correspondence](#). *Journal of Combinatorial Theory, Series A*, 26(3):308–313.
- Angeliki Giannou, Shashank Rajput, Jy-Yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. 2023. [Looped Transformers as Programmable Computers](#). In *Proceedings of the 40th International Conference on Machine Learning*, pages 11398–11442. PMLR.
- Jay R Goldman and Thomas Sundquist. 1992. [Lattice Path Enumeration by Formal Schema](#). *Advances in Applied Mathematics*, 13(2):216–251.
- Michael Hahn. 2020. [Theoretical Limitations of Self-Attention in Neural Sequence Models](#). *Transactions of the Association for Computational Linguistics*, 8:156–171.
- Yiding Hao, Dana Angluin, and Robert Frank. 2022. [Formal Language Recognition by Hard Attention Transformers: Perspectives from Circuit Complexity](#). *Transactions of the Association for Computational Linguistics*, 10:800–810.
- Nirmit Joshi, Gal Vardi, Adam Block, Surbhi Goel, Zhiyuan Li, Theodor Misiakiewicz, and Nathan Srebro. 2025. A Theory of Learning with Autoregressive Chain of Thought. *arXiv preprint arXiv:2503.07932*.
- Yanghao Li, Naiyan Wang, Jiaying Liu, and Xiaodi Hou. 2017. [Factorized Bilinear Models for Image Recognition](#). In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 2098–2106.
- Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. 2024. [Chain of Thought Empowers Transformers to Solve Inherently Serial Problems](#). In *The Twelfth International Conference on Learning Representations*.
- Charles London and Varun Kanade. 2026. Pause Tokens Strictly Increase the Expressivity of Constant-Depth Transformers. *Advances in Neural Information Processing Systems*, 38:89765–89795.
- Will Merrill and Ashish Sabharwal. 2026. A Little Depth Goes a Long Way: the Expressive Power of Log-Depth Transformers. *Advances in Neural Information Processing Systems*, 38:95315–95339.
- William Merrill and Ashish Sabharwal. 2024. [The Expressive Power of Transformers with Chain of Thought](#). In *The Twelfth International Conference on Learning Representations*.
- Franz Nowak, Anej Svete, Alexandra Butoi, and Ryan Cotterell. 2024. On the Representational Capacity of Neural Language Models with Chain-of-Thought Reasoning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12510–12548.
- Binghui Peng, Srini Narayanan, and Christos Papadimitriou. 2024. [On Limitations of the Transformer Architecture](#). In *First Conference on Language Modeling*.
- Jorge Pérez, Javier Marinković, and Pablo Barceló. 2019. On the Turing Completeness of Modern Neural Network Architectures. *arXiv preprint arXiv:1901.03429*.
- Jorge Pérez, Pablo Barceló, and Javier Marinkovic. 2021. [Attention is Turing-Complete](#). *Journal of Machine Learning Research*, 22(75):1–35.
- Ruizhong Qiu, Zhe Xu, Wenxuan Bao, and Hanghang Tong. 2025. Ask, and it shall be given: On the turing completeness of prompting. In *International Conference on Learning Representations*, volume 2025, pages 6286–6309.
- Michael Rizvi-Martel, Maude Lizaie, Clara Lacroce, and Guillaume Rabusseau. 2024. [Simulating Weighted Automata over Sequences and Trees with Transformers](#). In *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, volume 238, pages 2368–2376. PMLR.
- Clayton Sanford, Bahare Fatemi, Ethan Hall, Anton Tsitsulin, Mehran Kazemi, Jonathan Halcrow, Bryan Perozzi, and Vahab Mirrokni. 2024. Understanding Transformer Reasoning Capabilities via Graph Algorithms. *Advances in Neural Information Processing Systems*, 37:78320–78370.
- Clayton Sanford, Daniel J Hsu, and Matus Telgarsky. 2023. Representational strengths and limitations of transformers. In *Advances in Neural Information Processing Systems*, volume 36, pages 36677–36707.
- Arthur N. Strahler. 1957. [Quantitative Analysis of Watershed Geomorphology](#). *Transactions, American Geophysical Union*, 38(6):913–920.
- Lena Strobl, William Merrill, Gail Weiss, David Chiang, and Dana Angluin. 2024. [What Formal Languages Can Transformers Express? A Survey](#). *Transactions of the Association for Computational Linguistics*, 12:543–561.

- Xavier Gérard Viennot. 2002. A Strahler bijection between Dyck paths and planar trees. *Discrete Mathematics*, 246(1-3):317–329.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. 2022. [Chain of Thought Prompting Elicits Reasoning in Large Language Models](#). In *Advances in Neural Information Processing Systems*.
- Gail Weiss, Yoav Goldberg, and Eran Yahav. 2021. [Thinking Like Transformers](#). In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 11080–11090. PMLR.
- Gilad Yehudai, Noah Amsel, and Joan Bruna. 2025. [Compositional Reasoning with Transformers, RNNs, and Chain of Thought](#). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Haoyu Zhao, Yiming Duan, Junfeng Hao, and Sanjeev Arora. 2023. [Do Transformers Parse while Predicting the Masked Word?](#) In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 16513–16542. Association for Computational Linguistics.
- Hanlin Zhu, Shibo Hao, Zhiting Hu, Jiantao Jiao, Stuart Russell, and Yuandong Tian. 2026. [Emergence of Superposition: Unveiling the Training Dynamics of Chain of Continuous Thought](#). In *The Fourteenth International Conference on Learning Representations*.

A Related Works

A substantial body of work delineates what Transformers can compute in a single forward pass, for encoders (Hahn, 2020; Pérez et al., 2021; Weiss et al., 2021; Hao et al., 2022; Chiang et al., 2023; Sanford et al., 2023; Barceló et al., 2024) and decoders alike (Pérez et al., 2021; Merrill and Sabharwal, 2024; Peng et al., 2024; Barceló et al., 2025), surveyed by Strobl et al. (2024). The recurring conclusion is a ceiling: unique and generalized hard-attention encoders recognize only languages in AC^0 (Hao et al., 2022), which already excludes the Dyck languages. Later work sharpens the picture to exact automata-theoretic characterizations (Rizvi-Martel et al., 2024) and depth hierarchies (Merrill and Sabharwal, 2026). As such, it is fair to say that we work in the most restrictive of these regimes (unique hard attention, no layer normalization) and the expressive power we exhibit is attributable to CoT alone.

Building on Pérez et al.’s result that hard-attention decoders with unboundedly many steps compute any decidable language, Merrill and Sabharwal refine this into a step-counted hierarchy (log steps $\rightarrow L$, linear steps $\rightarrow NC^1$); Li et al. (2024) obtain constant-depth constructions for P/poly, Nowak et al. (2024) characterize CoT-augmented language models, and related gains are known for pause and filler tokens (London and Kanade, 2026) and continuous thought (Zhu et al., 2026). While these characterize classes, concrete problems instantiating them remain scarce. In this context, we reiterate Barceló et al.’s contribution in showing that the EH rank is exactly the minimum number of CoT steps for a single-layer hard-attention decoder. Since the EH rank of a binary decision tree obeys the same recurrence as its Strahler number (Dahiya and Mahajan, 2021), and computing the latter from a term representation is NC^1 -complete (Ganardi and Lohrey, 2026), branching complexity probes precisely the linear-step regime rather than being an arbitrary target.

A parallel thread constructs Transformers that execute procedures: looped architectures as programmable computers (Giannou et al., 2023), parsing during masked prediction (Zhao et al., 2023), universal simulation of attention (Dutta et al., 2025), and simulation of weighted automata over sequences and trees by bilinear maps (Rizvi-Martel et al., 2024), which we adopt as a primitive. For graph algorithms specifically, De Luca and Fountoulakis (2024) simulate DFS, BFS, Dijkstra, and Kosaraju by *looped* Transformers with graph-interacting heads and graph-size-independent parameter count; Sanford et al. (2024) chart architecture tradeoffs for graph reasoning. However, all of the above evade the CoT decoder model. Looping is not autoregressive generation, and neither line counts CoT steps. Within CoT, Zhu et al. (2026) study reachability (to identify which nodes in a graph can be reached from a specified source node) empirically without realizing the traversal, and Barceló et al. (2025) invoke exhaustive tree traversal as a proof device without constructing it or handling weights. We close this gap with explicit decoder constructions, making traversal an object of study rather than scaffolding. Notably, Ganardi and Lohrey’s own NC^1 upper bound proceeds by a heavy-subtree-first DFS, so a CoT realization of DFS is the natural bridge to their result.

On the tree–path correspondence end, while it admits several classical realizations (Viennot, 2002; Gall, 2005), its regularity is delicate even at the continuum (Evans, 2008), and none has previously been examined for CoT realizability.

B Missing Proofs of Lemmas and Propositions

Proof of Lemma 1. Suppose $\phi(T)$ is not a Dyck word for some $T \in \mathcal{T}_n$. Then for some edge $((i, s_i), (i + 1, s_{i+1}))$ on $\phi(T)$ labeled with D $s_{i+1} < 0$ or $s_{2(n-1)} \neq 0$. If the former is the case, there must be more -1 steps than $+1$ steps. If

this is the case, some edges in T are backtracked more than they are traversed, which is absurd. On the other hand, had $s_{2(n-1)} \neq 0$, specifically $s_{2(n-1)} > 0$, then we would not have backtracked some edges. This would, by definition, violate the DFS traversal scheme.

We verify $\psi(\phi(T)) = T$ by structural induction on n . The case $n = 1$ is clear: $\phi(T) = \varepsilon$ and $\psi(\varepsilon)$ is a lone root r . If the root has ordered children subtending subtrees $t_1, \dots, t_k$, then $\phi(t) = U\phi(t_1)D \cdots U\phi(t_k)D$. Reading this with ψ , the j -th block $U\phi(t_j)D$ creates a new child of the root and reconstructs t_j beneath it by the inductive hypothesis. The identity $\phi(\psi(w)) = w$ follows by the same argument on $|w|$. $\square$

Proof of Proposition 2. Since G_T and $\phi(G_T)$ are both finite, both $\text{ht}(G_T)$ and $\max_{0 \leq i \leq 2(n-1)} s_i$ are finite. Let $u_i \in V(G_T)$ denote the node at which the DFS pointer rests after the processing step i , so $u_0 = r$. We claim

$$s_i = \text{depth}(u_i) \quad (0 \leq i \leq 2(n-1)), \quad \text{and} \quad s_i - s_{i-1} = 1 \quad (1 \leq i \leq 2(n-1)). \quad (3)$$

The base case $s_0 = 0 = \text{depth}(r)$ is immediate. For the inductive step, a U at position i moves the pointer to a child of u_{i-1} , giving $\text{depth}(u_i) = \text{depth}(u_{i-1}) + 1 = s_{i-1} + 1 = s_i$. A D at position i moves it to the parent of u_{i-1} , giving $\text{depth}(u_i) = \text{depth}(u_{i-1}) - 1 = s_{i-1} - 1 = s_i$. This establishes (3).

Since DFS visits every node of G_T , the map $i \mapsto u_i$ is surjective onto $V(G_T)$. Together with (3),

$$\max_{0 \leq i \leq 2(n-1)} s_i = \max_{0 \leq i \leq 2(n-1)} \text{depth}(u_i) = \max_{v \in V(G_T)} \text{depth}(v) = \text{ht}(G_T). \quad \square$$

Proof of Proposition 3. Consider l with $1 \leq l \leq \text{ht}(G_T)$, and define

$$A_l := \{v \in V(G_T) : \text{depth}(v) = l\}, \\ B_l := \{i : s_i = l, s_i - s_{i-1} = 1\}.$$

We construct a bijection $\varphi_l : A_l \rightarrow B_l$. For each $v \in A_l$, DFS first arrives at v from its parent via a unique U step; let i_v denote the index of this step. By Proposition 2, $s_{i_v} = \text{depth}(u_{i_v}) = \text{depth}(v) = l$, and by construction $s_{i_v} - s_{i_v-1} = 1$, so $i_v \in B_l$. Set $\varphi_l(v) := i_v$.

Injectivity. DFS visits each node for the first time at a distinct step, so $v \neq w$ implies $i_v \neq i_w$.

Surjectivity. Every $i \in B_l$ is a U step satisfying $s_i = l$, so the pointer moves to u_i with $\text{depth}(u_i) = l$ by Proposition 2. Since DFS first enters any node via a downward step, and step i is the first occasion u_i is reached (a second descent to u_i would require a prior ascent from it, which would have been recorded as the unique D step terminating that visit), we have $i = i_{u_i} = \varphi_l(u_i)$.

Hence $|A_l| = |B_l|$ for every $1 \leq l \leq \text{ht}(G_T)$. It remains to verify that excluding the level $l = 0$ does not alter the maximum. The root is the unique node at depth 0, so $|A_0| = 1$. If $\text{wd}(G_T) = 1$, then $|A_l| = 1$ for all l , and since $\text{ht}(G_T) \geq 1$ for $n \geq 2$ the set $\{A_l : 1 \leq l \leq \text{ht}(G_T)\}$ is non-empty with maximum 1 = $\text{wd}(G_T)$. If $\text{wd}(G_T) \geq 2$, the maximum is attained at some $l \geq 1$ and $|A_0| = 1$ is strictly dominated. In both cases,

$$\text{wd}(G_T) = \max_{0 \leq l \leq \text{ht}(G_T)} |A_l| = \max_{1 \leq l \leq \text{ht}(G_T)} |A_l| = \max_{0 < l \leq \text{ht}(G_T)} |B_l|. \quad \square$$

Proof of Correctness of Algorithm 1. We first address two implicit invariants. First, the branch on line 8 is unreachable when $i = 0$: if the current character were D at depth zero, the lattice path would descend below zero, contradicting the non-negativity condition of a Dyck path. Second, whenever the algorithm reaches line 8, it is guaranteed that a unique $j < nl$ satisfying $A_w[j, nl] = 1$ exists. Indeed, a step D at position t implies $s_t - s_{t-1} = -1$, so the path is closing a previously opened step U ; the corresponding upward transition was handled by line 6 at the time that U was processed, which set $A_w[i, nl] \leftarrow 1$ for the unique parent i of nl at that moment. The uniqueness of such j follows from the observation that node nl is introduced exactly once, at the single U step that executes line 6 for nl , so no index $j' \neq j$ can satisfy $A_w[j', nl] = 1$ at the time line 8, since every previous step U operates on a strictly lower value of nl . Finally, ψ is injective: if $\psi(w) = \psi(w')$ for two Dyck words $w, w' \in \mathcal{D}_{m-1}$, then applying ϕ to both sides and invoking the Lemma 1 yields $w = \phi(\psi(w)) = \phi(\psi(w')) = w'$. Hence, the algorithm terminates correctly on every valid Dyck word, the output A_w is the adjacency matrix of $\psi(w)$, and ψ is a well-defined injection in $\mathcal{D}_{m-1}$. $\square$

C Preliminaries

Fact 1. Let $v \in \mathbb{R}^n$ be an input vector, $W \in \mathbb{R}^{m \times n}$ be a weight matrix, and $c \in \mathbb{R}^m$ be a bias vector. An affine transformation defined by $v' = Wv + c$ can be rewritten strictly as a linear transformation $v' = W'\tilde{v}$ by concatenating the input vector v with c .

Proof. If we explicitly want to define our new input vector by concatenating v and c , we can write $\tilde{v} = [v^\top \parallel c^\top]^\top \in \mathbb{R}^{n+m}$.

Define $W' \in \mathbb{R}^{m \times (n+m)}$ as $W' = [W \parallel I_m]$. Then

$$\begin{aligned} W'(\tilde{v}) &= [W \parallel I_m] \begin{bmatrix} v \\ c \end{bmatrix} = Wv + I_m c \\ &= Wv + c = v'. \end{aligned} \quad \square$$

Fact 2. Let X and U be vector spaces over a field $\mathbb{R}$. A linear transformation $L : X \rightarrow U$ parameterized by a matrix $W \in \mathbb{R}^{m \times n}$ such that $L(v) = Wv$ can be represented as a bilinear operation.

Proof. To represent the linear transformation as a bilinear-style operation where the matrix weights are treated as parameters and v is the input, we consider the second argument as $\mathbb{R}$.

We define a mapping $B : X \times \mathbb{R} \rightarrow U$, as $B(v, c) = Wv \cdot c$, where $v \in V$, $c \in \mathbb{R}$. To show this behaves linearly with respect to the input vector space X , let $v_1, v_2 \in X$ be input vectors, and let $\alpha, \beta \in \mathbb{R}$ be scalars.

$$\begin{aligned} B(\alpha v_1 + \beta v_2, c) &= W(\alpha v_1 + \beta v_2) \cdot c \\ &= (\alpha Wv_1 + \beta Wv_2) \cdot c \\ &= \alpha(Wv_1 \cdot c) + \beta(Wv_2 \cdot c) \\ &= \alpha B(v_1, c) + \beta B(v_2, c). \end{aligned}$$

This satisfies linearity in the first argument. Similarly for the second coordinate

$$\begin{aligned} B(v, \alpha c_1 + \beta c_2) &= Wv(\alpha c_1 + \beta c_2) \\ &= Wv\alpha c_1 + Wv\beta c_2 \\ &= \alpha Wv \cdot c_1 + \beta Wv \cdot c_2 \\ &= \alpha B(v, c_1) + \beta B(v, c_2). \end{aligned} \quad \square$$

Lemma 4. Let U be a vector space and let U', U'' be vector spaces with $W = U' \oplus U''$. Let $B_1 : U \times U \rightarrow U'$ and $B_2 : U \times U \rightarrow U''$ be bilinear maps, and define $B : U \times U \rightarrow W$ by

$$B(v, w) = \begin{pmatrix} B_1(v, w) \\ B_2(v, w) \end{pmatrix}.$$

Then B is bilinear.

Proof. We verify linearity in the first argument; the second is identical. Let $v, \tilde{v}, w \in U$ and let c be a scalar.

By the bilinearity of B_1 and B_2 in their first argument,

$$B_i(v + \tilde{v}, w) = B_i(v, w) + B_i(\tilde{v}, w), \quad i \in \{1, 2\}.$$

Since addition in $W = U' \oplus U''$ is taken componentwise,

$$\begin{aligned} B(v + \tilde{v}, w) &= \begin{pmatrix} B_1(v, w) + B_1(\tilde{v}, w) \\ B_2(v, w) + B_2(\tilde{v}, w) \end{pmatrix} \\ &= \begin{pmatrix} B_1(v, w) \\ B_2(v, w) \end{pmatrix} + \begin{pmatrix} B_1(\tilde{v}, w) \\ B_2(\tilde{v}, w) \end{pmatrix} \\ &= B(v, w) + B(\tilde{v}, w). \end{aligned}$$

By the homogeneity of B_1 and B_2 in their first argument, $B_i(cv, w) = c B_i(v, w)$, and since scalar multiplication in W is componentwise,

$$\begin{aligned} B(cv, w) &= \begin{pmatrix} c B_1(v, w) \\ c B_2(v, w) \end{pmatrix} = c \begin{pmatrix} B_1(v, w) \\ B_2(v, w) \end{pmatrix} \\ &= c B(v, w). \end{aligned}$$

The second argument is treated identically, using the bilinearity of each B_i there. Hence B is bilinear. $\square$

Before we present the explicit constructions we substantiate the [Definition 3.1](#).

Realization of Algorithms via Chain-of-Thought. The construction of Transformers that implement functions and algorithmic dynamics has been extensively studied in the expressivity literature ([Giannou et al., 2023](#); [Zhao et al., 2023](#); [Dutta et al., 2025](#)). To ground the constructions presented in [section 2](#), [Definition 3.1](#) outline the general scheme we adopt. The input sequence $(x_0, \dots, x_{m-1}) \in (\mathbb{R}^d)^m$ encodes the graph structure, where each x_i may additionally carry a fixed number of auxiliary coordinates reserved for intermediate computation. The initial token $y_0 \in \mathbb{R}^d$ serves as the designated start symbol for each algorithm; for instance, in graph traversal, a fixed block of coordinates within y_0 encodes the identity of the source vertex. However, during the design of individual algorithms, including those that take the lattice path as input, we will specify the encoding required to realize particular algorithms via CoT.

We call the realization *uniform* if the map from n to the family of resultant decoders is logspace-computable. In our work, all constructions follow uniformity in this sense. Note that the encoding enc represents an interpretation of the state of the algorithm $\mathcal{A}$. While enc is not spelled out in each construction, its role is implicit and can be read from context. For instance, in the Dijkstra construction the block π_{Dist} carries the distance Dist , which serves as $\text{enc}(s_t)$; by contrast, in the DFS construction only the order of traversal matters, and no such $\text{enc}(s_t)$ is present. The remaining cases follow the same pattern.

D DFS

Theorem 5. *A two-layer, two-head Transformer decoder can simulate the depth-first traversal DFS on any simple directed graph $G = (V, E, A)$ in $O(|V| + |E|)$ chain-of-thought steps.*

Proof. We show that the t^{th} step of a two-layer, two-head Transformer decoder implements $\text{dfs}^{(t)}(s)$. Let $G = (V, E, A)$ be a graph with adjacency matrix $A \in \{0, 1\}^{n \times n}$, and denote by $e_i \in \{0, 1\}^n$ the standard basis vector corresponding to vertex $v_i \in V$.

The initial token sequence consists of embedded vectors $x_0, \dots, x_{n-1} \in \mathbb{R}^d$ with $d = 6n + 3$, preceded by a sentinel token $x_{\perp} = 0^d$. Each embedding is a concatenation of nine blocks, that together capture the instantaneous description of the traversal: the current vertex, the parent vertex, the visited marker, the neighborhood, a temporary register, a buffer register, two state-control flags, and a type flag. We define the node embedding $x_i \in \mathbb{R}^{6n+3}$ for each $v_i \in V$ as

$$x_i = [e_i \parallel 0^n \parallel 0^n \parallel A_{i,:} \parallel 0^n \parallel 0^n \parallel -1 \parallel -1 \parallel 0]^{\top},$$

where the block $A_{i,:}$ is the i^{th} row of the adjacency matrix, encoding the neighborhood $\mathcal{N}(v_i)$. Note that the collection $\{x_i\}_{i=0}^{n-1}$ uniquely identifies G . The embedding carries three scalar flags. A state-control flag $flg_1 \in \{1, 0, -1\}$ indicates whether the current step is a *traverse*, *backtrack*, or *idle* operation, respectively. An auxiliary flag flg_2 supports the final computation of flg_1 . A type flag $flg_3 \in \{0, 1\}$ distinguishes x_i s ($flg_3 = 0$) from y_t s ($flg_3 = 1$). The dynamic DFS at step t is maintained in vector $y_t \in \mathbb{R}^{6n+3}$, where $t \leq O(|V| + |E|)$. Given a source vertex $s = v_p \in V$, the initial state y_0 is

$$y_0 = [e_p \parallel 0^n \parallel e_p \parallel A_{p,:} \parallel 0^n \parallel 0^n \parallel -1 \parallel -1 \parallel 1]^{\top}.$$

The two embeddings differ in precisely two blocks: y_0 records e_p in the visited restriction and sets $flg_3 = 1$, reflecting that s has been visited and that y_0 is a traversal token.

Let $\mathcal{E} \subset \{0, 1\}^n$ denote the set of standard basis vectors in $\mathbb{R}^n$. For notational convenience, we define the following selectors from $\mathbb{R}^d$ to access individual blocks of an embedding. The maps $\pi_{cur}, \pi_{par} : \mathbb{R}^d \rightarrow \mathcal{E}$ extract the current and parent node indicators⁵; the maps $\pi_{vis}, \pi_{nbr} : \mathbb{R}^d \rightarrow \{0, 1\}^n$ extract the visited and neighborhood vectors; the maps $\pi_{tmp}, \pi_{buf} : \mathbb{R}^d \rightarrow \mathbb{R}^n$ extract the temporary and buffer registers; and $\pi_{flg_k} : \mathbb{R}^d \rightarrow \mathbb{R}$ for $k \in \{1, 2, 3\}$ extracts the corresponding control flag.

We proceed inductively. Suppose y_t encodes the instantaneous description of G immediately before $\text{DFS}^{(t+1)}(s)$. The first operation updates the control flags $\pi_{flg_1}(y_t)$ and $\pi_{flg_2}(y_t)$; the attention in this layer is trivial, with $W_O^{(1)} = \mathbf{0}^{d \times d}$. The update is carried out by the FFN block. The bilinear map $W_1^{(1)}$ computes the inner product $\gamma_t = (\mathbf{1} - \pi_{vis}(y_t))^{\top} \pi_{nbr}(y_t)$, which counts the number of unvisited neighbors of the current vertex, together with its unit-shifted counterpart $\gamma_t - 1$, placing these into the flg_1 and flg_2 positions respectively. After the ReLU activation, the subsequent linear map $W_2^{(1)}$ writes $\pi_{flg_1}(y_t) = \text{ReLU}(\gamma_t) - \text{ReLU}(\gamma_t - 1)$, $\pi_{flg_2}(y_t) = 0$, and copies $\pi_{cur}(y_t)$ into the buffer register. We denote the resulting vector by $y_t^{(1)}$. Observe that $\pi_{flg_1}(y_t^{(1)})$ is 1 precisely

⁵At initialization, $\pi_{par}(\alpha) = 0^n$ for every $\alpha \in \{x_0, \dots, x_{n-1}, y_0\}$, reflecting that $\text{par}(v_i) = \phi$ for all $v_i \in V \cup \{s\}$.

when $\gamma_t \geq 1$ and 0 otherwise, encoding the *traverse-or-backtrack* decision. After this step, all preceding vectors $\{x_\perp^{(1)}, x_0^{(1)}, \dots, x_{n-1}^{(1)}, y_0^{(1)}, \dots, y_{t-1}^{(1)}\}$ differ from their initial values in the flg_1 , flg_2 , and buffer positions. Because none of these positions of the aforementioned vector interact with any operation in the subsequent layer, this modification does not affect the computations.

The second layer comprises two mutually exclusive attention heads. The first head executes the operation *traverse*. The query $Q_T^{(2)} y_t^{(1)}$ is a bilinear map (Rizvi-Martel et al., 2024) implementing the Hadamard product $(\mathbf{1} - \pi_{vis}(y_t^{(1)})) \odot \pi_{nbr}(y_t^{(1)})$, which masks out all previously visited vertices from the neighborhood. Evaluated against keys $K_T^{(2)} \alpha = \pi_{cur}(\alpha)$ for all $\alpha \in \{x_\perp^{(1)}, x_0^{(1)}, \dots, x_{n-1}^{(1)}, y_0^{(1)}, \dots, y_t^{(1)}\}$, the UHA resolves the lowest-indexed token α'_T (among $\{x_i^{(1)}\}$) whose current-node encoding $\pi_{cur}(\alpha'_T)$ corresponds to $\text{dfs}^{(t+1)}(s)$. When $\pi_{flg_1}(y_t^{(1)}) = 0$, α'_T defaults to $x_\perp^{(1)}$.

The second head implements the operation *backtrack*. The query $Q_B^{(2)} y_t^{(1)}$ is a bilinear map computing $(1 - \pi_{flg_1}(y_t^{(1)})) \cdot [\pi_{par}(y_t^{(1)}) \parallel \pi_{flg_3}(y_t^{(1)})]$, which activates only when $\pi_{flg_1}(y_t^{(1)}) = 0$. Evaluated against keys $K_B^{(2)} \alpha = [\pi_{cur}(\alpha) \parallel \pi_{flg_3}(\alpha)]$ for all $\alpha \in \{x_\perp^{(1)}, x_0^{(1)}, \dots, x_{n-1}^{(1)}, y_0^{(1)}, \dots, y_t^{(1)}\}$, the UHA resolves the lowest-indexed token α'_B (among $\{y_i^{(1)}\}$) such that $\pi_{cur}(\alpha'_B)$ corresponds to the vertex first visited during $\text{dfs}^{(t')}(s)$ for some $0 \leq t' < t$. When $\pi_{flg_1}(y_t^{(1)}) = 1$, the query vanishes and α'_B defaults to $x_\perp^{(1)}$. The linear map $W_O^{(2)} \in \{-1, 0, 1\}^{d \times 2d}$ produces the combined update $\alpha'' = \alpha'_T + \alpha'_B$, where

$$\begin{aligned} \alpha''_T &= [-\pi_{cur}(\alpha'_T) \parallel -\pi_{par}(\alpha'_T) \parallel \pi_{cur}(\alpha'_T) \parallel 0^n \parallel \pi_{nbr}(\alpha'_T) \parallel 0^n \parallel 0^3]^\top \quad \text{and} \\ \alpha''_B &= [-\pi_{cur}(\alpha'_B) \parallel -\pi_{par}(\alpha'_B) \parallel 0^{2n} \parallel \pi_{nbr}(\alpha'_B) \parallel 0^n \parallel 0^3]^\top. \end{aligned}$$

Since the two heads are mutually exclusive – governed by $\pi_{flg_1}(y_t^{(1)})$ – exactly one of α''_T and α''_B is 0^{6n+3} .

Since G is simple, during *traverse* the current vertex and any unvisited neighbor are necessarily distinct, whence $\pi_{cur}(y_t^{(1)})^\top \pi_{cur}(\alpha''_T) = 0$. Similarly, during *backtrack*, $\pi_{par}(y_t^{(1)})^\top \pi_{par}(\alpha''_B) = 0$ holds strictly: were it otherwise, we would have $\pi_{cur}(\alpha''_B) = \pi_{par}(\alpha''_B)$, implying a self-loop and contradicting the assumption that G is simple.

The linear map $W_1^{(2)}$ then negates the current and parent restrictions of $\alpha'' + y_t^{(1)}$, resets the temporary register, and copies the (previous) temporary register into the neighborhood position, leaving all other positions intact. The subsequent ReLU activation discards the negated priors: $-\pi_{cur}(y_t^{(1)})$ is eliminated from the current restriction in both traversal and backtrack, while $-\pi_{par}(y_t^{(1)})$ is eliminated from the parent restriction during backtrack. Denote the result by β . At this stage, β agrees with the desired output y_{t+1} of the $(t+1)$ -th DFS dynamic in every restriction except the parent restriction during *traverse*. To resolve this, the bilinear map $W_2^{(2)}$ sets $\pi_{par}(\beta)$ to value $\pi_{flg_1}(\beta)\pi_{buf}(\beta) + (1 - \pi_{flg_1}(\beta))\pi_{par}(\beta)$, which selects the buffered vertex (the pre-traversal current node) when $\pi_{flg_1}(y_t) = 1$ and retains the existing parent otherwise. The chain-of-thought terminates when $\pi_{cur}(y_{t+1}) = 0^n$. $\square$

E Dijkstra Algorithm

Theorem 6. A two-layer, single-head Transformer decoder can simulate the Dijkstra algorithm Dkst on any simple connected directed graph $G = (V, E, A)$ with $A = \mathbb{R}_{>0}$ in exactly $|V| - 1$ chain-of-thought steps.

Proof. We show that the t^{th} step of a two-layer single-head Transformer decoder implements $\text{Dkst}^{(t)}$. Let $G = (V, E, A)$ be a graph with adjacency matrix $A \in \mathbb{R}_{>0}^{n \times n}$, fix a source $s = v_p \in V$, and write $e_i \in \{0, 1\}^n$ for the standard basis vector corresponding to v_i . The input token sequence consists of embeddings $x_0, \dots, x_{n-1} \in \mathbb{R}^d$ with $d = 5n + 1$ that together encode the instantaneous description of the traversal: the identity of the current vertex, the tentative distance from s to that vertex, the visited indicator, the outgoing edge weights, the global distance vector Dist , and a buffer. Concretely, the embedding of vertex v_i is

$$x_i = [e_i \parallel -\lambda \parallel 0^n \parallel A_{i,:} \parallel \lambda^n \parallel 0^n]^\top,$$

where $A_{i,:}$ is the i^{th} row of the adjacency matrix encoding the outgoing weights from v_i , and λ is a sufficiently large constant. The scalar $-\lambda$ in the second block serves as a sentinel indicating that no finite shortest-path estimate has yet been assigned; the fourth block will be repurposed during output token generation, and the final block acts as a

scratchpad. Note that the collection $\{x_i\}_{i=0}^{n-1}$ uniquely determines G . The dynamic Dkst at step $t \in \{0, \dots, n-1\}$ is maintained in a vector $y_t \in \mathbb{R}^d$; given source $s = v_p$, the initial state is

$$y_0 = [e_p \parallel 0 \parallel e_p \parallel 0^n \parallel \lambda^n \parallel 0^n]^\top.$$

That is, y_0 starts the CoT with the initialized state $\text{Dkst}^{(0)}$ – source visited, distance zero – whereas the embeddings $\{x_i\}$ encode the graph structure. Let $\mathcal{E} \subset \{0, 1\}^n$ denote the set of standard basis vectors in $\mathbb{R}^n$. For notational convenience, we define the following selectors that extract individual blocks from any embedding in $\mathbb{R}^d$: $\pi_{cur} : \mathbb{R}^d \rightarrow \mathcal{E}$, $\pi_{crd} : \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0} \cup \{-\lambda\}$, $\pi_{vis} : \mathbb{R}^d \rightarrow \{0, 1\}^n$, $\pi_{wei}, \pi_{dis} : \mathbb{R}^d \rightarrow \mathbb{R}^n$, and $\pi_{buf} : \mathbb{R}^d \rightarrow \mathbb{R}^n$. In the generated sequence $\{y_{t+1}\}$, the scalar $\pi_{crd}(y_{t+1})$ records $\text{Dist}^{(t+1)}[j']$ for the uniquely newly visited vertex, i.e., the vertex $v_{j'}$ satisfying $vst^{(t+1)}(v_{j'}) = 1$ and $vst^{(t)}(v_{j'}) = 0$. The block $\pi_{wei}(y_{t+1}) = 0^n$ is not used to carry adjacency information in the output tokens and is instead repurposed as another buffer for storing intermediate results between layers. Note that the codomain $\mathbb{R}_{\geq 0} \cup \{-\lambda\}$ reflects a dual role: on input tokens $\{x_i\}$ the selector π_{crd} returns the sentinel $-\lambda$, while on generated tokens $\{y_t\}$ it returns a non-negative shortest-path distance.

We proceed inductively. Suppose y_t encodes the instantaneous description of G immediately prior to $\text{Dkst}^{(t+1)}$; the first layer updates the distance vector from $\text{Dist}^{(t)}$ to $\text{Dist}^{(t+1)}$. The query $Q^{(1)}y_t$ extracts $\pi_{cur}(y_t)$, the one-hot identity of the current vertex, and is evaluated against keys $K^{(1)}\alpha = \pi_{cur}(\alpha)$ for every token $\alpha \in \{x_0, \dots, x_{n-1}, y_0, \dots, y_t\}$; the UHA selects the lowest-indexed match, which is necessarily the input embedding x_i corresponding to $cur^{(t)} = v_i$, thereby getting access of the adjacency row $A_{i,:}$. Taking $V^{(1)}$ as identity, the output projection $W_O^{(1)}$ zeroes out all blocks except π_{wei} , so that after the residual connection the weight block carries $A_{i,:}$ – in particular, this is where π_{wei} transitions from its role as a zero buffer on output tokens to carrying adjacency data within the layer. Let this vector be α'' . The first linear map of the feed-forward network, $W_1^{(1)}$, then computes $\pi_{dis}(\alpha'' + y_t)[i] - \pi_{crd}(\alpha'' + y_t) - \pi_{wei}(\alpha'' + y_t)[i]$ for each $i \in \{0, \dots, n-1\}$; after the ReLU activation, π_{wei} stores the non-negative updates corresponding to $\text{Dist}^{(t+1)}$ with all other positions intact. Denote the result by β and observe that $\pi_{wei}(\beta) \geq 0^n$. The second linear map $W_2^{(1)}$ replaces $\pi_{dis}(\beta)$ with $\pi_{dis}(\beta) - \pi_{wei}(\beta)$ and resets $\pi_{wei}(\beta)$ to 0^n . We denote the resulting vector by $y_t^{(1)}$.

The second layer selects the unvisited vertex of minimum tentative distance. The query $Q^{(2)}$ is a bilinear map (Rizvi-Martel et al., 2024) realizing the Hadamard product $(1 - \pi_{vis}(y_t^{(1)})) \odot (\lambda - \pi_{dis}(y_t^{(1)}))$, which assigns each unvisited vertex v_j a score of $\lambda - \pi_{dis}(y_t^{(1)})[j] > 0$. Evaluated against keys $K^{(2)}\alpha = \pi_{cur}(\alpha)$ for all $\alpha \in \{x_0^{(1)}, \dots, x_{n-1}^{(1)}, y_0^{(1)}, \dots, y_t^{(1)}\}$, UHA selects the lowest-indexed token α' among $\{x_i^{(1)}\}_{i=0}^{n-1}$ at which $-\pi_{dis}(y_t^{(1)})$ is maximized over unvisited vertices – equivalently, the vertex $v_{j'}$ such that $j' = \text{argmin}\{\text{Dist}^{(t+1)}[j] \mid vst^{(t)}(v_j) = 0\}$. The output projection $W_O^{(2)}$ produces

$$\alpha'' = [-\pi_{cur}(\alpha') \parallel 0 \parallel \pi_{cur}(\alpha') \parallel 0^n \parallel 0^n \parallel \pi_{cur}(\alpha')]^\top,$$

so that the residual connection yields $\pi_{cur}(\alpha'' + y_t^{(1)}) = \pi_{cur}(y_t^{(1)}) - \pi_{cur}(\alpha')$ and $\pi_{vis}(\alpha'' + y_t^{(1)}) = \pi_{vis}(y_t^{(1)}) + \pi_{cur}(\alpha')$, while π_{buf} now holds $\pi_{cur}(\alpha')$. We claim $\pi_{cur}(\alpha')^\top \pi_{cur}(y_t^{(1)}) = 0$: were this not so, the current vertex would be attending to itself, implying a self-loop and contradicting the assumption that G is simple. Hence, the residual updates the current block to the difference of two orthogonal one-hot vectors and augments the visited set by exactly one element. The feed-forward network of the second layer completes the transition. The linear map $W_1^{(2)}$ acts on $\alpha'' + y_t^{(1)}$ at two blocks while leaving all others intact: it negates the current block and sets the buffer to $\gamma \in \mathbb{R}^n$ defined coordinate-wise by

$$\gamma[i] = \pi_{dis}(\alpha'' + y_t^{(1)})[i] + \lambda \pi_{buf}(\alpha'' + y_t^{(1)})[i] - \lambda \sum_{j=0}^{n-1} \pi_{buf}(\alpha'' + y_t^{(1)})[j],$$

for each $i \in \{0, \dots, n-1\}$. Since

$$\pi_{buf}(\alpha'' + y_t^{(1)}) = \pi_{cur}(\alpha') = e_{j'},$$

the sum equals unity and the expression simplifies: at coordinate j' the λ terms cancel, yielding $\gamma[j'] = \text{Dist}^{(t+1)}[j']$, while at every other coordinate $\gamma[i] = \text{Dist}^{(t+1)}[i] - \lambda < 0$ for λ sufficiently large. The subsequent ReLU therefore retains $\text{Dist}^{(t+1)}[j']$ at the j' th coordinate of the buffer and zeros out all others, while simultaneously eliminating $-\pi_{cur}(y_t^{(1)})$ from the current block, leaving $\pi_{cur}(\alpha')$ as the sole survivor. Denote the result by $y_t^{(1)'}.$ Finally, $W_2^{(2)}$ sets $\pi_{crd}(y_{t+1}) = \sum_{i=0}^{n-1} \pi_{buf}(y_t^{(1)'})[i] = \text{Dist}^{(t+1)}[j']$ and resets $\pi_{buf}(y_{t+1}) = 0^n$, resulting y_{t+1} . After exactly $n-1$ transitions the visited indicator satisfies $\pi_{vis}(y_{n-1}) = 1^n$, which serves as the halting criterion. $\square$

F Realizing Algorithm 1

Theorem 7. A single-layer, two-head Transformer decoder can simulate the Algorithm 1 in the $2(m-1)$ CoT-steps.

Proof. The t^{th} decoding step reconstructs the partial tree built after reading the lattice path $\{(\tau, s_\tau)\}_{\tau=0}^t$, which is one iteration of the for loop on line 3. Let $\mathcal{E}_m = \{e_0, \dots, e_{m-1}\}$ and $\mathcal{E}_{2m-2} = \{\hat{e}_0, \dots, \hat{e}_{2m-3}\}$ denote the standard bases of $\mathbb{R}^m$ and $\mathbb{R}^{2m-2}$. The construction maintains i , nl , and A_w across iterations, encoded in eight blocks of dimension $d = m^2 + 6m$: the current vertex cur (the iterator i of line 2), the label nl of the most recently created vertex, the parent par recorded when an U creates a vertex, the flattened adjacency matrix A of the partial tree, the cursor pos marking the position read so far, two scratch blocks stp holding $q_\tau \in \{+1, -1\}$ for U resp. D and on a CoT token y_t is 0, taking a transient value in $\{+1, -1\}$ only while a step is being read; and buf holding the parent encoding retrieved on a D (so that line 9 is realized), and a flag flg separating input tokens x ($flg = 0$) from CoT tokens y ($flg = 1$).

The input tokens $x_0, \dots, x_{2m-3} \in \mathbb{R}^d$ encode the Dyck word $w = w_0 \dots w_{2m-3}$ of the path $\{(\tau, s_\tau)\}_{\tau=0}^{2m-2}$:

$$x_\tau = [0^m \parallel 0^m \parallel 0^m \parallel 0^{m^2} \parallel \hat{e}_\tau \parallel q_\tau \parallel 0^m \parallel 0]^T, \\ q_\tau = s_{\tau+1} - s_\tau.$$

The dynamic is carried out by tokens $y_t \in \mathbb{R}^d$, with

$$y_0 = [e_0 \parallel e_0 \parallel 0^m \parallel 0^{m^2} \parallel \hat{e}_0 \parallel 0 \parallel 0^m \parallel 1]^T,$$

so that $i = nl = 0$ (cur, nl at e_0) and par, A_w are empty.

For $v \in \{0, 1\}$ and $q \in \{+1, -1\}$,

$$\text{ReLU}(v + q - 1) = \mathbb{1}[q = +1]v, \quad \text{ReLU}(v - q - 1) = \mathbb{1}[q = -1]v, \quad (4)$$

since $\text{ReLU}(v + 1 - 1) = v$ while $\text{ReLU}(v - 1 - 1) = \text{ReLU}(v - 2) = 0$, and symmetrically. Hence, one ReLU selects a branch by the sign of q_t and passes a binary v unchanged on the selected branch, zeroing it on the other. Like other constructions, we use the selectors

$$\begin{aligned} \pi_{cur}, \pi_{nl} : \mathbb{R}^d &\rightarrow \mathcal{E}_m, \\ \pi_{par}, \pi_{buf} : \mathbb{R}^d &\rightarrow \mathcal{E}_m \cup \{0^m\}, \\ \pi_A : \mathbb{R}^d &\rightarrow \{0, 1\}^{m^2}, \\ \pi_{pos} : \mathbb{R}^d &\rightarrow \mathcal{E}_{2m-2}, \\ \pi_{stp} : \mathbb{R}^d &\rightarrow \{+1, -1, 0\}, \\ \text{and } \pi_{flg} : \mathbb{R}^d &\rightarrow \{0, 1\}, \end{aligned}$$

each extracting its block from an embedding in $\mathbb{R}^d$.

We argue by induction, with the hypothesis that $\pi_A(y_t)$ holds the adjacency matrix of the partial tree built after reading $x_0, \dots, x_{t-1}$. Step t is driven by two heads, one per branch of line 4: the U head is selected when $\pi_{stp}(x_t) = +1$, the D head when $\pi_{stp}(x_t) = -1$.

The U head reads the current symbol. With query $Q_U y_t = \pi_{pos}(y_t)$ and keys $K_U \alpha = \pi_{pos}(\alpha)$ over $\alpha \in \{x_0, \dots, x_{2m-3}, y_0, \dots, y_t\}$, the score $\pi_{pos}(y_t)^\top \pi_{pos}(\alpha)$ is 1 exactly on the tokens whose block pos is $\hat{e}_t$, namely x_t and y_t ; the lowest-indexed of these is the input token $x_t =: \alpha'_U$. Its value $\alpha''_U := V_U \alpha'_U$ copies $\pi_{stp}(\alpha'_U) = q_t$ into the stp block and is 0 elsewhere.

The D head retrieves a parent. With query $Q_D y_t = [\pi_{cur}(y_t) \parallel \pi_{flg}(y_t)]$ and keys $K_D \alpha = [\pi_{cur}(\alpha) \parallel \pi_{flg}(\alpha)]$, the score is 2 on the y tokens whose current block equals $\pi_{cur}(y_t)$ and at most 1 on every other token; among these the lowest-indexed is the creation token of the current vertex i_t (the y token that first set $\pi_{cur} = \pi_{cur}(y_t)$ on a U step), which precedes y_t and is thus selected over it. Call it α'_D . Its value $\alpha''_D := V_D \alpha'_D$ copies the recorded parent $\pi_{par}(\alpha'_D)$ into the buf block and is 0 elsewhere.

The projection $W_O \in \{0, 1\}^{d \times 2d}$ sums the two contributions, $\alpha'' = \alpha''_U + \alpha''_D$.

We now construct the FFN. Write $\beta = \alpha'' + y_t$ for its input; since the heads write only the stp and buf blocks, β agrees with y_t on every other block, so $\pi_{cur}(\beta) = \pi_{cur}(y_t)$, $\pi_{nl}(\beta) = \pi_{nl}(y_t)$, $\pi_A(\beta) = \pi_A(y_t)$, while $\pi_{stp}(\beta) = q_t$ and $\pi_{buf}(\beta) = \pi_{par}(\alpha'_D)$. Let $h = W_1 \beta \in \mathbb{R}^{d'}$ with $d' = 2m^2 + 6m - 2 > d$, and overload π'_i for the blocks of h :

$$h = [\pi'_{cur} \parallel \pi'_{nl} \parallel \pi'_{par} \parallel \pi'_{buf} \parallel \pi'_A \parallel \pi'_{pos} \parallel \pi'_{A'}],$$

which drops the flg and stp blocks and adds A' , the indicator of the edge inserted on a U (line 6). Let $S \in \{0, 1\}^{m \times m}$ be the shift $s_{ij} = \mathbb{1}[j = i - 1]$ (so $Se_k = e_{k+1}$ and $Se_{m-1} = 0$); addition of a scalar to a vector is broadcast, i.e. added to every coordinate. By (4), W_1 sets each block so that the activation ReLU realizes the branch of line 4:

$$\begin{aligned}\pi'_{cur}(h) &= S\pi_{nl}(\beta) + \pi_{stp}(\beta) - 1 \\ \pi'_{nl}(h) &= \pi_{nl}(\beta) - \pi_{stp}(\beta) - 1 \\ \pi'_{par}(h) &= \pi_{cur}(\beta) + \pi_{stp}(\beta) - 1 \\ \pi'_{buf}(h) &= \pi_{buf}(\beta) - \pi_{stp}(\beta) - 1 \\ \pi'_A(h) &= \pi_A(\beta) \\ \pi'_{A'}(h)_{am+b} &= \beta^\top M^{(a,b)}\beta + \pi_{stp}(\beta) - 1 \\ &= \pi_{cur}(\beta)_a \cdot \pi_{nl}(\beta)_{b-1} + \pi_{stp}(\beta) - 1 \\ \pi'_{pos}(h) &= \pi_{pos}(\beta),\end{aligned}$$

for $(a, b) \in \{0, \dots, m-1\}^2$, where $M^{(a,b)} \in \mathbb{R}^{d \times d}$ is the constant matrix with $\beta^\top M^{(a,b)}\beta = \pi_{cur}(\beta)_a \cdot \pi_{nl}(\beta)_{b-1}$, and $\pi_{nl}(\beta)_{b-1}$ is read as 0 when $b = 0$.

Recall $\pi_{stp}(\beta) = q_t \in \{+1, -1\}$. By (4), after activation $\pi'_{cur}(h')$ equals the shifted label $S\pi_{nl}(\beta)$ on a U and 0 on a D, while $\pi'_{buf}(h')$ equals the retrieved parent on a D and 0 on a U; the remaining blocks gate analogously. Let $S' \in \{0, 1\}^{(2m-2) \times (2m-2)}$ be the shift $S'\hat{e}_\tau = \hat{e}_{\tau+1}$ with $S'\hat{e}_{2m-3} = 0$. With $h' = \text{ReLU}(h)$, the map $W_2 \in \{0, 1\}^{d \times d'}$ produces $y_{t+1} = W_2 h'$ by summing the gated blocks:

$$\begin{aligned}\pi_{cur}(y_{t+1}) &= \pi'_{cur}(h') + \pi'_{buf}(h'), \\ \pi_{nl}(y_{t+1}) &= \pi'_{cur}(h') + \pi'_{nl}(h'), \\ \pi_{par}(y_{t+1}) &= \pi'_{par}(h'), \\ \pi_A(y_{t+1})[am+b] &= \pi'_A(h')[am+b] \\ &\quad + \pi'_{A'}(h')[am+b] + \pi'_{A'}(h')[bm+a], \\ \pi_{pos}(y_{t+1}) &= S'\pi'_{pos}(h'), \quad \pi_{stp}(y_{t+1}) = 0, \\ \pi_{buf}(y_{t+1}) &= 0^m, \quad \pi_{flg}(y_{t+1}) = 1.\end{aligned}$$

On a U step this gives $\pi_{cur}(y_{t+1}) = \pi_{nl}(y_{t+1}) = e_{nl_t+1}$ and sets $A[i_t, nl_t+1] = A[nl_t+1, i_t] = 1$ with $\pi_{par}(y_{t+1}) = e_{i_t}$, realizing lines 5–6; on a D step it gives $\pi_{cur}(y_{t+1}) = \pi_{buf}(h') = e_{\text{par}(i_t)}$ while π_{nl} and π_A are unchanged, realizing line 9. In both cases the cursor advances, $\pi_{pos}(y_{t+1}) = \hat{e}_{t+1}$. $\square$

G Realizing st on Trees and Paths

Theorem 8. *A four-layer, two-head Transformer decoder can compute $\text{st}(G_T)$, the Strahler number of a tree G_T during the simulation of DFS in exactly $2n - 1$ CoT steps, where n denotes the number of vertices in G_T .*

Proof. We show that the t^{th} step of a four-layer, two-head decoder, extending that of Theorem 5 in its embedding and by two further layers, computes the Strahler number $\text{st}(G_T)$ of the tree G_T during DFS traversal. We will follow Observation 2.

We extend the embedding from $\mathbb{R}^{6n+3}$ to $\mathbb{R}^{6n+14}$ by appending ten scalar blocks, written $[\dots \parallel M \parallel c \parallel M_{buf} \parallel c_{buf} \parallel flg_4 \parallel tmp_{D-1} \parallel tmp_D \parallel tmp_{D+1} \parallel tmp_{c_{buf}-1} \parallel tmp_{c_{buf}} \parallel tmp_{c_{buf}+1}]$. In a generated token y_t , the block M holds the running maximum of the Strahler numbers of the already-finished children of the current vertex, and $c \in \{0, 1, 2\}$ counts how many of those children attain M . Within a *backtrack* step these blocks are transformed by the layers below before the invariant is restored in y_{t+1} . The blocks M_{buf} and c_{buf} are transient buffers, zero on every input and generated token and nonzero only within a step during a *backtrack*. The flag $flg_4 \in \{0, 1, \dots, 2n-1\}$ is a timestamp, equal to 0 on every input token and on y_0 and to t on the generated token y_t , so that the generated tokens are linearly ordered by flg_4 . The five blocks tmp are scratch positions used by the later layers; their role is given where they first arise. Whereas the type flag flg_3 separates inputs from generated tokens by value in $\{0, 1\}$, the flag flg_4 records their order; supplied to the backtrack head of Theorem 5, it converts that head's leftmost selection into a rightmost one, so that the head resolves the most recent rather than the first occurrence of the parent vertex, as made precise below.

The selectors of [Theorem 5](#) are retained, and we add $\pi_M, \pi_c, \pi_{M_{buf}}, \pi_{c_{buf}}, \pi_{flg_4}$ and $\pi_{tmp} : \mathbb{R}^d \rightarrow \mathbb{R}$ with $d = 6n + 14$. The input tokens are those of the traversal with the new blocks set to zero,

$$x_i = [e_i \parallel 0^n \parallel 0^n \parallel A_{i,:} \parallel 0^n \parallel 0^n \parallel -1 \parallel -1 \parallel 0 \parallel 0^{11}]^\top,$$

so that $\pi_M(x_i) = \pi_c(x_i) = \pi_{M_{buf}}(x_i) = \pi_{c_{buf}}(x_i) = \pi_{flg_4}(x_i) = 0$ for every $v_i \in V$, and the sentinel remains $x_\perp = 0^d$. Given the source $s = v_p$, the initial state is

$$y_0 = [e_p \parallel 0^n \parallel e_p \parallel A_{p,:} \parallel 0^n \parallel 0^n \parallel -1 \parallel -1 \parallel 1 \parallel 0^{11}]^\top,$$

which agrees with the traversal's initial state on the inherited blocks and carries an empty accumulator $\pi_M(y_0) = \pi_c(y_0) = 0$, empty buffers $\pi_{M_{buf}}(y_0) = \pi_{c_{buf}}(y_0) = 0$, and timestamp $\pi_{flg_4}(y_0) = 0$. We take the Strahler number of a leaf to be zero.

The selection effected by the backtrack head is the following. With respect to the t^{th} query Qy_t , the scalar $a_{i,t}$ is the attention score of the key against the token indexed i among $\{x_\perp, x_0, \dots, x_{n-1}, y_0, \dots, y_t\}$. Let $j_0 < j_1 < \dots < j_p$ denote the indices, in ascending order, that jointly attain the maximum score. Where unique hard attention selects the smallest such index j_0 , the inclusion of flg_4 in the key makes the head select the largest, j_p , the most recently generated token among those of maximal score.

The induction follows that of [Theorem 5](#), recalled only to the extent the Strahler computation requires. Suppose y_t encodes the instantaneous description immediately before $\text{DFS}^{(t+1)}(s)$. The first layer leaves its attention trivial and, through the feed-forward block, writes into $\pi_{flg_1}(y_t)$ the indicator $\text{ReLU}(\gamma_t) - \text{ReLU}(\gamma_t - 1)$ of whether the current vertex has an unvisited neighbour, where $\gamma_t = (\mathbf{1} - \pi_{vis}(y_t))^\top \pi_{nbr}(y_t)$, so that $\pi_{flg_1}(y_t^{(1)})$ is 1 on a *traverse* and 0 on a *backtrack*; it clears π_{flg_2} and copies $\pi_{cur}(y_t)$ into the buffer. The eleven new blocks are untouched by this layer. We denote the result $y_t^{(1)}$.

In the second layer the traverse head acts exactly as in [Theorem 5](#), resolving the next unvisited neighbour on a *traverse* and defaulting to $x_\perp^{(1)}$ on a *backtrack*. The backtrack head retains its query but is keyed to select the parent's most recent occurrence rather than its first. Its query is the bilinear map

$$Q_B^{(2)} y_t^{(1)} = (1 - \pi_{flg_1}(y_t^{(1)})) \cdot [\pi_{par}(y_t^{(1)}) \parallel \pi_{flg_3}(y_t^{(1)})],$$

which vanishes on a *traverse* and otherwise presents the parent indicator together with the scalar $\pi_{flg_3}(y_t^{(1)}) = 1$. It is evaluated against the keys

$$K_B^{(2)} \alpha = [\lambda \cdot \pi_{cur}(\alpha) \parallel \pi_{flg_4}(\alpha)],$$

for $\alpha \in \{x_\perp^{(1)}, x_0^{(1)}, \dots, x_{n-1}^{(1)}, y_0^{(1)}, \dots, y_t^{(1)}\}$, where the constant $\lambda \geq 2n - 1$ exceeds every attainable timestamp. The resulting score $\lambda \cdot \pi_{par}(y_t^{(1)})^\top \pi_{cur}(\alpha) + \pi_{flg_4}(\alpha)$ is lexicographic: the term $\lambda \cdot \pi_{par}(y_t^{(1)})^\top \pi_{cur}(\alpha)$ places every occurrence of the parent strictly above every non-occurrence, since λ alone exceeds any timestamp, while the constant $\pi_{flg_3}(y_t^{(1)}) = 1$ admits $\pi_{flg_4}(\alpha)$ into the score with unit weight. The head thus resolves the token α'_B of greatest timestamp among the occurrences of the parent, that is, the parent as it was last left. The query supplies π_{flg_3} , fixed at 1, rather than its own $\pi_{flg_4} = t$, which is common to all keys and so cannot order the parent occurrences.

The initial token y_0 does not reach the backtrack head. Unless G is the null graph, the root s has an unvisited neighbor, so $\gamma_0 \geq 1$ and $\pi_{flg_1}(y_0^{(1)}) = 1$; the backtrack query vanishes and y_0 is served by the traverse head.

We now specify the value carried by the two heads. On the inherited blocks they act as in [Theorem 5](#); we extend only the backtrack value, and only on the blocks M and c . Let α'_B be the token resolved by the backtrack head. Its value map $V_B^{(2)}$ moves the accumulator into the buffers and clears the originals,

$$\begin{aligned} \pi_{M_{buf}}(V_B^{(2)} \alpha'_B) &= \pi_M(\alpha'_B), \quad \pi_{c_{buf}}(V_B^{(2)} \alpha'_B) = \pi_c(\alpha'_B), \\ \pi_M(V_B^{(2)} \alpha'_B) &= 0, \quad \pi_c(V_B^{(2)} \alpha'_B) = 0, \end{aligned}$$

and is the identity on every other coordinate. The traverse value α''_T is zero on all ten new blocks, so the buffers are written by the backtrack head alone, and M, c, flg_4 by neither head.

Form $\beta = \alpha''_T + \alpha'_B + y_t^{(1)}$. On a *backtrack* the backtrack query does not vanish; the buffers receive the parent accumulator while the head contributes zero to M, c :

$$\begin{aligned} \pi_{M_{buf}}(\beta) &= \pi_M(\alpha'_B), \quad \pi_{c_{buf}}(\beta) = \pi_c(\alpha'_B), \\ \pi_M(\beta) &= \pi_M(y_t^{(1)}), \quad \pi_c(\beta) = \pi_c(y_t^{(1)}), \quad \pi_{flg_4}(\beta) = t. \end{aligned}$$

On a *traverse* the query vanishes, α'_B defaults to $x_{\perp}^{(1)} = 0^d$, and the buffers stay empty, $\pi_{M_{buf}}(\beta) = \pi_{c_{buf}}(\beta) = 0$, while M, c pass through from $y_t^{(1)}$. On a *backtrack* the pair $(\pi_M(\beta), \pi_c(\beta))$ is the accumulator of the vertex being left, v say, and the buffers hold the accumulator of its parent, u say.

Prior to define the parameters $W_1^{(2)}$, and $W_2^{(2)}$ along with the subsequent layers, we set up the proof for defining the *tmp* blocks. Let $D = \text{st}(v) - M_u$ where $M_u = \pi_{M_{buf}}(\beta)$. Similarly, $c_v = \pi_c(\beta)$ and $c_u = \pi_{c_{buf}}(\beta)$. With this we want

$$\pi_M(y_{t+1}) = M_u + \text{ReLU}(\text{st}(v) - M_u) + \mathbb{1}[D = 0] \mathbb{1}[c_u > 0] \quad (5)$$

$$\pi_c(y_{t+1}) = \mathbb{1}[D > 0]c_u + \mathbb{1}[D = 0] \mathbb{1}[c_u = 0] + \mathbb{1}[D < 0]1 \quad (6)$$

during *backtrack*.

The linear map $W_1^{(2)}$ sets

$$\pi_c(W_1^{(2)}\beta) = \pi_c(\beta) - 1,$$

$$\pi_{flg_4}(W_1^{(2)}\beta) = \pi_{flg_4}(\beta) + 1,$$

$$\pi_{tmp_{c_{buf}-1}}(W_1^{(2)}\beta) = \pi_{c_{buf}}(\beta) - 1,$$

leaving the other blocks intact; after the activation we write the result β' . Since $c_v \in \{0, 1, 2\}$, the rectified count is $\pi_c(\beta') = \text{ReLU}(c_v - 1) = \mathbb{1}[c_v \geq 2]$. The linear map $W_2^{(2)}$ then forms $y_t^{(2)}$ by

$$\pi_M(y_t^{(2)}) = \pi_M(\beta') + \pi_c(\beta') = M_v + \mathbb{1}[c_v \geq 2] = \text{st}(v),$$

and, writing $\pi_M(\beta') + \pi_c(\beta') - \pi_{M_{buf}}(\beta') = \text{st}(v) - M_u = D$,

$$\pi_{tmp_{D-1}}(y_t^{(2)}) = D - 1,$$

$$\pi_{tmp_D}(y_t^{(2)}) = D,$$

$$\pi_{tmp_{D+1}}(y_t^{(2)}) = D + 1.$$

After the second layer the block M carries $\text{st}(v)$, and the blocks $tmp_{D-1}, tmp_D, tmp_{D+1}$ carry $D - 1, D, D + 1$.

The third and fourth layers leave the attention trivial, $W_O^{(3)} = W_O^{(4)} = \mathbf{0}$, and act through their feed-forward blocks alone. The linear map $W_1^{(3)}$ extends $y_t^{(2)}$ by six scalar blocks, $1 - D, -D, -D - 1, 1 - c_u, -c_u$, and $-c_u - 1$. The activation reads off indicators through the integer identities

$$\mathbb{1}[v = 0] = \text{ReLU}(v + 1) - 2 \text{ReLU}(v) + \text{ReLU}(v - 1)$$

$$\mathbb{1}[v > 0] = \text{ReLU}(v) - \text{ReLU}(v - 1).$$

write the rectified vector β'' . The linear map $W_2^{(3)}$ then produces $y_t^{(3)} \in \mathbb{R}^d$, returning the indicators to the *tmp* blocks:

$$\begin{aligned} \pi_M(y_t^{(3)}) &= \pi_M(\beta'') + \pi_{-D}(\beta'') \\ &= \max(\text{st}(v), M_u), \end{aligned}$$

$$\pi_{tmp_{D-1}}(y_t^{(3)}) = \mathbb{1}[D > 0],$$

$$\pi_{tmp_D}(y_t^{(3)}) = \mathbb{1}[D = 0],$$

$$\pi_{tmp_{D+1}}(y_t^{(3)}) = \mathbb{1}[D < 0],$$

$$\pi_{tmp_{c_{buf}}}(y_t^{(3)}) = \mathbb{1}[c_u > 0],$$

$$\pi_{tmp_{c_{buf}-1}}(y_t^{(3)}) = \mathbb{1}[c_u = 0].$$

The 1s the indicators require are supplied by $\pi_{flg_3} = 1$. The fourth layer carries out the accumulation. Recall that after the third layer the blocks $tmp_{D-1}, tmp_D, tmp_{D+1}$ hold $\mathbb{1}[D > 0], \mathbb{1}[D = 0], \mathbb{1}[D < 0]$ and the blocks $tmp_{c_{buf}}, tmp_{c_{buf}-1}$ hold $\mathbb{1}[c_u > 0], \mathbb{1}[c_u = 0]$. The bilinear map $W_1^{(4)}$ sets

$$\pi_M(W_1^{(4)}y_t^{(3)}) = \pi_M(y_t^{(3)}) + \pi_{tmp_D}(y_t^{(3)})\pi_{tmp_{c_{buf}}}(y_t^{(3)}),$$

$$\pi_c(W_1^{(4)}y_t^{(3)}) = \pi_{tmp_{D-1}}(y_t^{(3)})1 + \pi_{tmp_D}(y_t^{(3)})\pi_{tmp_{c_{buf}-1}}(y_t^{(3)}) + \pi_{tmp_{D+1}}(y_t^{(3)})\pi_{c_{buf}}(y_t^{(3)}),$$

which, by the contents of those blocks, are $\pi_M(y_t^{(3)}) + \mathbb{1}[D = 0] \mathbb{1}[c_u > 0]$ and $\mathbb{1}[D > 0]c_u + \mathbb{1}[D = 0] \mathbb{1}[c_u = 0] + \mathbb{1}[D < 0]1$, realizing (5) and (6). Each is a product of two blocks of $y_t^{(3)}$, hence of degree two. The map further resets M_{buf} , c_{buf} , and the tmp . blocks to zero, leaving the remaining coordinates intact; write the result β''' .

The bilinear map $W_2^{(4)}$ corrects M and c for the *traverse* case, assigning

$$\pi_M(y_{t+1}) = (1 - \pi_{flg_1}(y_t^{(3)}))\pi_M(\beta'''), \quad (7)$$

$$\pi_c(y_{t+1}) = (1 - \pi_{flg_1}(y_t^{(3)}))\pi_c(\beta'''), \quad (8)$$

and sets $flg_2 = 0$, $flg_3 = 1$. On a *backtrack* the gate is 1 and the accumulator commits; on a *traverse* it is 0, so the freshly visited vertex receives the empty accumulator $(0, 0)$. Each coordinate is a product of two blocks, hence of degree two. This is y_{t+1} .

At the $(2n - 1)^{\text{th}}$ step the current restriction becomes 0^n ; this final *backtrack* folds s under the same invariant, giving $\pi_M(y_{2n-1}) = \text{st}(s) = \text{st}(G_T)$ $\square$

Lemma 9. *Let $d \geq 2$, and let $v = [m \parallel h \parallel \mathbf{0}_d]^\top \in \mathbb{R}^{2d+1}$ be an input vector where $m \in \{-1, +1\}$, $h \in \{0, 1\}^d$ is a one-hot vector, and $\mathbf{0}_d$ is the zero vector in $\mathbb{R}^d$. Let $h' \in \{0, 1\}^d$ denote the zero-padded, one-position shift of h directed by the sign of m (right-shifted if $m = +1$, left-shifted if $m = -1$). There exists an FFN with ReLU activation $W_2 \text{ReLU}(W_1 v + b_1) + b_2$, that produces $[m \parallel h \parallel h']^\top$.*

Proof. Let I_d be the $d \times d$ identity matrix, $\mathbf{1}_d$ be the d -dimensional all-ones vector, and $\mathbf{0}_{r \times c}$ be the $r \times c$ zero matrix. Define the right-shift matrix $S_R \in \mathbb{R}^{d \times d}$ such that $(S_R)_{i,j} = 1$ if $i - j = 1$ and 0 otherwise, and let the left-shift matrix be $S_L = S_R^\top$.

Construct the first linear projection $W_1 \in \mathbb{R}^{(3d+2) \times (2d+1)}$ and bias $b_1 \in \mathbb{R}^{3d+2}$ to partition the hidden representation into five blocks:

$$W_1 = \begin{pmatrix} 1 & \mathbf{0}_d^\top & \mathbf{0}_d^\top \\ -1 & \mathbf{0}_d^\top & \mathbf{0}_d^\top \\ \mathbf{0}_d & I_d & \mathbf{0}_{d \times d} \\ 0.5\mathbf{1}_d & S_R & \mathbf{0}_{d \times d} \\ -0.5\mathbf{1}_d & S_L & \mathbf{0}_{d \times d} \end{pmatrix}, \quad b_1 = \begin{bmatrix} 0 \\ 0 \\ \mathbf{0}_d \\ -0.5\mathbf{1}_d \\ -0.5\mathbf{1}_d \end{bmatrix}$$

Applying $\text{ReLU}(W_1 v + b_1)$ yields the hidden vector $z = [z_{m,+}, z_{m,-}, z_h, z_+, z_-]^\top$:

- Scalars $z_{m,+} = \text{ReLU}(m)$ and $z_{m,-} = \text{ReLU}(-m)$ isolate the sign of m .
- Since $h \in \{0, 1\}^d$ is non-negative, $z_h = \text{ReLU}(h) = h$.
- The shift gates resolve to mutually exclusive activations: if $m = +1$, $z_+ = S_R h$ and $z_- = \mathbf{0}_d$; if $m = -1$, $z_+ = \mathbf{0}_d$ and $z_- = S_L h$.

Construct the second linear projection $W_2 \in \mathbb{R}^{(2d+1) \times (3d+2)}$ and bias $b_2 = \mathbf{0}_{2d+1}$ as:

$$W_2 = \begin{pmatrix} 1 & -1 & \mathbf{0}_d^\top & \mathbf{0}_d^\top & \mathbf{0}_d^\top \\ \mathbf{0}_d & \mathbf{0}_d & I_d & \mathbf{0}_{d \times d} & \mathbf{0}_{d \times d} \\ \mathbf{0}_d & \mathbf{0}_d & \mathbf{0}_{d \times d} & I_d & I_d \end{pmatrix}$$

Multiplying $W_2 z + b_2$ sums the mutually exclusive shift gates and reconstructs the input blocks:

$$v' = \begin{bmatrix} z_{m,+} - z_{m,-} \\ I_d z_h \\ I_d z_+ + I_d z_- \end{bmatrix} = \begin{bmatrix} m \\ h \\ h' \end{bmatrix}. \quad \square$$

Lemma 10. *For a real bound $\lambda \geq 0$, and let $v = [m \parallel M \parallel c]^\top \in \{-1, 1\} \times [0, \lambda] \times [0, \lambda]$ be the input vector. There exists an FFN with ReLU activation $W_2 \text{ReLU}(W_1 v + b_1) + b_2$, such that the output vector satisfies the following:*

$$v' = \begin{cases} [1 \parallel 0 \parallel 0]^\top & \text{if } m = 1, \\ [-1 \parallel M \parallel c]^\top & \text{if } m = -1. \end{cases}$$

Proof. For any scalar $x \in [0, \lambda]$ and a variable $m \in \{-1, 1\}$, define the scalar gating function:

$$g(m, x) = \text{ReLU}(x) - \text{ReLU}\left(x + \frac{\lambda}{2}m - \frac{\lambda}{2}\right).$$

When $m = 1$, the second term simplifies to $\text{ReLU}(x)$, yielding $g(1 \parallel x) = \text{ReLU}(x) - \text{ReLU}(x) = 0$. When $m = -1$, the second term becomes $\text{ReLU}(x - \lambda) = 0$ because $x \leq \lambda$, leaving $g(-1 \parallel x) = \text{ReLU}(x) = x$ since $x \geq 0$. Furthermore, the identity mapping for the binary variable m can be expressed exactly as $m = \text{ReLU}(m) - \text{ReLU}(-m)$.

We construct $h = \text{ReLU}(W_1 v + b_1)$ by defining the first layer's parameters as:

$$W_1 = \begin{pmatrix} 1 & 0 & 0 \\ -1 & 0 & 0 \\ 0 & 1 & 0 \\ \frac{\lambda}{2} & 1 & 0 \\ 0 & 0 & 1 \\ \frac{\lambda}{2} & 0 & 1 \end{pmatrix}, \quad b_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -\frac{\lambda}{2} \\ 0 \\ -\frac{\lambda}{2} \end{bmatrix}.$$

Applying the ReLU activation yields the hidden state vector:

$$h = \text{ReLU} \begin{bmatrix} m \\ -m \\ M \\ M + \frac{\lambda}{2}m - \frac{\lambda}{2} \\ c \\ c + \frac{\lambda}{2}m - \frac{\lambda}{2} \end{bmatrix}.$$

Setting the output bias to zero ($b_2 = \mathbf{0}$) and defining the output projection matrix W_2 as:

$$W_2 = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & -1 \end{pmatrix},$$

we obtain the network output:

$$\mathcal{N}(v) = \begin{bmatrix} \text{ReLU}(m) - \text{ReLU}(-m) \\ g(m \parallel M) \\ g(m \parallel c) \end{bmatrix} = \begin{bmatrix} m \\ g(m \parallel M) \\ g(m \parallel c) \end{bmatrix}.$$

Evaluating this vector at $m = 1$ yields $[1 \parallel 0 \parallel 0]^\top$, and evaluating at $m = -1$ yields $[-1 \parallel M \parallel c]^\top$. $\square$

Theorem 11. A four-layer, single-head Transformer decoder can compute $\text{st}(\psi(w))$, the Strahler number of a tree corresponding to the Dyck word w in exactly n CoT steps, where $n = |w|$.

Proof Sketch. Like other proofs, we show that the t^{th} step of a four-layer single head decoder can calculate $\text{st}(\psi(w))$. The proof will rely on the following observation and [Observation 2](#).

Observation 3. Let the Dyck word $w = w_0 \cdots w_{n-1}$ encode the lattice path $\{(\tau, s_\tau)\}_{\tau=0}^n$. Let $q_\tau = s_\tau - s_{\tau-1}$. For a time step t , $\text{st}(\psi(w_0 \cdots w_t))$ depends on two (M, c) -pairs (see [Observation 2](#)) if $q_t = -1$. The first is the (M_v, c_v) -pair of the vertex v with co-ordinate (t, s_t) , while the second (M_u, c_u) is associated with the vertex u with co-ordinate $(t', s_{t'})$ such that $\nexists t'', t' < t'' < t$ and $s_{t''} = s_{t'}$.

Based on this, we shall provide the construction up to the second layer alone and the last layer partially. The proof thereon follows [Theorem 8](#). Assume $\mathcal{E}_n = \{e_0, \dots, e_{n-1}\}$ denotes the standard basis vector on $\mathbb{R}^n$. Let $S \in \{0, 1\}^{n \times n}$ be the shift $s_{ij} = \mathbb{1}[j = i - 1]$ (so $S e_k = e_{k+1}$ and $S e_{n-1} = 0$). The input tokens $x_0, \dots, x_{n-1} \in \mathbb{R}^d$ where $d = 3n + 15$ encodes the lattice paths $\{(\tau, s_\tau)\}_{\tau=0}^n$ as follows:

$$x_\tau = [e_\tau \parallel \tau + 1 \parallel 0 \parallel q_\tau \parallel 0 \parallel e_{s_\tau} \parallel 0^n \parallel 0^4 \parallel 0 \parallel 0^6]^\top,$$

for $\tau = 0$ to $n - 1$. The Dynamic is carried out by tokens $y_t \in \mathbb{R}^d$ with

$$y_0 = [e_0 \parallel 1 \parallel 0 \parallel 1 \parallel 0 \parallel e_0 \parallel 0^n \parallel 0^4 \parallel 1 \parallel 0^6]^\top.$$

The selectors

$$\begin{aligned}
&\pi_{pos} : \mathbb{R}^d \rightarrow \mathbb{R}^n, \\
&\pi_{pos_{sc}}, \pi_{tmp_{pos_{sc}}}, \pi_{tmp_q} : \mathbb{R}^d \rightarrow \mathbb{R}, \\
&\pi_q : \mathbb{R}^d \rightarrow \{-1, +1\}, \\
&\pi_{ht}, \pi_{tmp_{ht}} : \mathbb{R}^d \rightarrow \mathbb{R}^n, \\
&\pi_M, \pi_c, \pi_{M_{buf}}, \pi_{c_{buf}} : \mathbb{R}^d \rightarrow \mathbb{R}, \\
&\pi_{flg} : \mathbb{R}^d \rightarrow \{0, 1\}, \\
&\pi_{tmp_{D-1}}, \pi_{tmp_D}, \pi_{tmp_{D+1}}, \pi_{tmp_{c_{buf}-1}}, \pi_{tmp_{c_{buf}}}, \\
&\text{and } \pi_{tmp_{c_{buf}+1}} : \mathbb{R}^d \rightarrow \mathbb{R},
\end{aligned}$$

extract the relevant blocks from an embedding in $\mathbb{R}^d$. The first three blocks correspond to the one-hot position τ , its decimal equivalent, and a temporary block. The blocks denoted by q and tmp_q register q_τ and its temporary computation trace. The following two store the one-hot equivalent of s_τ and an intermediate buffer. The blocks M and c correspond to the M_v and c_v of (2). Similarly, M_{buf} and c_{buf} correspond to M_u and c_u . The block flg distinguishes the input tokens x_i s ($flg = 0$) from the generated tokens y_i s ($flg = 1$), while tmp_{flg} is for temporary calculation. The remaining blocks continue to serve the purpose as in [Theorem 8](#). We argue that $\text{st}(\psi(w)) = \pi_M(y_n) + \mathbb{1}[\pi_c(y_n)]$.

The first layer will update the blocks pos , q , and ht . The query extracts the block pos and shifts the one $Q^{(1)}y_t = S\pi_{pos}(y_t)$ and evaluated against keys $K\alpha = \pi_{pos}(\alpha)$ for all $\alpha \in \{x_0, x_1, \dots, x_{n-1}, y_0, \dots, y_t\}$. The UHA selects the lowest-indexed match from x_i 's, say α' . The vector after value projection is α'' . Then $\pi_{pos}(\alpha'') = -\pi_{pos}(\alpha')$, $\pi_{tmp_{pos_{sc}}}(\alpha'') = \pi_{pos_{sc}}(\alpha')$, $\pi_{tmp_q}(\alpha'') = \pi_q(\alpha')$, and, leaving rest to 0. Consider $W_O^{(1)}$ being the identity, note that the residual connection in $\alpha'' + y_t$ does not affect the non-zero values of y_t in any of its blocks. The linear projection $W_1^{(1)}$ performs two operations. It transforms the block into $-\pi_{pos}(\alpha'' + y_t)$ and fulfills the role of W_1 in [Lemma 9](#), where the blocks tmp_q , h , and tmp_h are treated as placeholders for m , h , and h' , respectively. After activation ReLU, the block pos will register $S\pi_{pos}(y_t)$. Followed by $W_2^{(1)}$, the block tmp_h stores the one-hot encoding of $s_{\tau+1}$. Let the resulting representation be denoted by $y_t^{(1)}$. It is important to emphasize that, with the exception of the block flg , all updates and value assignments within the constituent blocks of $y_t^{(1)}$ are carried out in auxiliary blocks. This ensures that the actual blocks associated with the preceding tokens in the layered computation remain unchanged throughout the intermediate computational stages. Since flg does not participate in both the query and key computations of the remaining layers, it remains unaffected by these operations.

The subsequent layers work by guessing that $q_{\tau+1}$ is a D (-1) step. However, the last layer applies [Lemma 10](#)⁶ to perform the final correction in M and c blocks depending on the value of the block q in $y_t^{(4)}$. The second layer will realize the observation we made earlier in this proof – finding the appropriate token (among y_i s $i < t$) such that $\pi_{ht}(y_i) = \pi_{tmp_{ht}}(y_t^{(1)})$ and $\nexists i', i < i' < t$ and $\pi_{ht}(y_{i'}) = \pi_{tmp_{ht}}(y_t^{(1)})$. Let the query be $Q^{(2)}y_t^{(1)} = [\pi_{tmp_{ht}}(y_t^{(1)}) \parallel \pi_{flg}(y_t^{(1)})]$ and the key be $K^{(2)}\alpha = [\lambda\pi_{ht}(\alpha) \parallel \pi_{pos_{sc}}(\alpha)]$ for all $\alpha \in \{x_0^{(1)}, x_1^{(1)}, \dots, x_{n-1}^{(1)}, y_0^{(1)}, \dots, y_t^{(1)}\}$ and $\lambda > n + 1$. Let this vector be α' . Clearly $\pi_{ht}(y_t^{(1)})^\top \pi_{tmp_{ht}}(y_t^{(1)}) = 0$ ensures that we obtain a right-most $i < t$. The value projection $V^{(2)}$ then

$$\begin{aligned}
&\pi_{M_{buf}}(V^{(2)}\alpha') = \pi_M(\alpha'), \quad \pi_{c_{buf}}(V^{(2)}\alpha') = \pi_c(\alpha'), \\
&\pi_M(V^{(2)}\alpha') = 0, \quad \pi_c(V^{(2)}\alpha') = 0.
\end{aligned}$$

The subsequent operations will be analogous to that of [Theorem 8](#) that realizes [Observation 2](#) using the blocks M , c , M_{buf} , c_{buf} , tmp_{D-1} , tmp_D , tmp_{D+1} , $tmp_{c_{buf}-1}$, $tmp_{c_{buf}}$, $tmp_{c_{buf}+1}$, and flg in place of flg_3 of [Theorem 8](#). However, in the fourth layer we employ [Lemma 10](#) that in its FFN block by dissolving the gating (7) in $W_2^{(4)}$. This final projection $W_2^{(4)}$ produces y_{t+1} by nullifying all temporary positions along with the buffers M_{buf} and c_{buf} . Finally, $\text{st}(\psi(w)) = \pi_M(y_n) + \mathbb{1}[\pi_c(y_n) \geq 2]$. $\square$

It should be noted that the pair $(t', s_{t'})$ in [Observation 3](#) plays a pivotal role in transmitting the Strahler rank to the current vertex. In the absence of this contribution, the pair (M_v, c_v) alone would not suffice to determine the Strahler rank at the current vertex (see [Figure 3](#)).

⁶On the corresponding tree of $n' = \frac{n}{2} + 1$ nodes, the Strahler number is at most $\log_2(n' + 2)$, so any λ above this threshold satisfies the lemma.

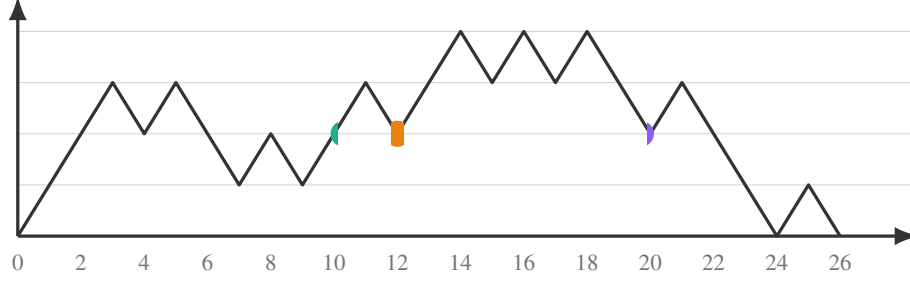


Figure 3: As illustrated in Figure 2, calculating the Strahler number at the point $(24, 2)$ without incorporating the specific pair $(t', s_{t'}) = (16, 2)$ (from Observation 3) yields a value equal to that evaluated at $(20, 2)$ in the above path. This would incorrectly suggest a generation-invariant property of the Strahler number for previously explored children.

H Realizing wd on Trees and Paths

Lemma 12. Let $v = [v_1 \parallel v_2 \parallel v_3]^\top \in \mathbb{R}^3$ be an input vector such that $v_1 \in \{0, 1\}$, and $v_2, v_3 \in \mathbb{Z}^+$ are bounded such that $v_2 \leq \lambda$ for a large λ . Let the target transformation $f : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ be defined piecewise as:

$$f(v) = \begin{cases} [0 \parallel v_2 + 1 \parallel \max(v_3, v_2 + 1)]^\top & \text{if } v_1 = 0 \\ [1 \parallel 1 \parallel \max(v_3, 1)]^\top & \text{if } v_1 = 1 \end{cases}$$

It can be exactly computed by a single hidden-layer FFN utilizing a ReLU activation function with a hidden dimension of 4, taking the form $f(v) = W_2 \text{ReLU}(W_1 v + b_1) + b_2$.

Proof. To construct the exact FFN, let $\lambda \in \mathbb{R}$ be a strictly positive large scalar.

Define the weight matrix $W_1 \in \mathbb{R}^{4 \times 3}$ and bias vector $b_1 \in \mathbb{R}^4$:

$$W_1 = \begin{pmatrix} 1 & 0 & 0 \\ -\lambda & 1 & 0 \\ 0 & 0 & 1 \\ -\lambda & 1 & -1 \end{pmatrix}, \quad b_1 = [0 \parallel 0 \parallel 0 \parallel 1]^\top$$

Evaluating the pre-activation state $z = W_1 v + b_1$ yields $z = [v_1 \parallel v_2 - \lambda v_1 \parallel v_3 \parallel v_2 - v_3 - \lambda v_1 + 1]^\top$.

Let $h = \text{ReLU}(z) = \max(0, z)$. Because $v_1 \in \{0, 1\}$ and $v_3 \geq 1$, the first and third components are strictly non-negative and pass through unchanged ($h_1 = v_1, h_3 = v_3$). To evaluate the conditional components for both valid states of v_1 :

Case $v_1 = 0$: The second term is v_2 , and the fourth term is $v_2 - v_3 + 1$. Thus, the hidden state is $h = [0 \parallel v_2 \parallel v_3 \parallel \max(0, v_2 - v_3 + 1)]^\top$.

Case $v_1 = 1$: The second term becomes $v_2 - \lambda$. Since $\lambda > \lambda \geq v_2$, this is strictly negative and clamps to zero. The fourth term becomes $v_2 - v_3 - \lambda + 1$. Since $v_2 - \lambda + 1 \leq 0$ and $v_3 \geq 1$, this entire expression is strictly negative and also clamps to zero. Thus, the hidden state is $h = [1 \parallel 0 \parallel v_3 \parallel 0]^\top$.

The final projection $W_2 \in \mathbb{R}^{3 \times 4}$ and output bias $b_2 \in \mathbb{R}^3$:

$$W_2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}, \quad b_2 = [0 \parallel 1 \parallel 0]^\top$$

Applying this final projection verifies the transformation exactly maps to the conditions of $f(v)$:

- For $v_1 = 0$:

$$\begin{aligned} f(v) &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix} \begin{bmatrix} 0 \\ v_2 \\ v_3 \\ \max(0, v_2 - v_3 + 1) \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 \\ v_2 + 1 \\ v_3 + \max(0, v_2 - v_3 + 1) \end{bmatrix} = \begin{bmatrix} 0 \\ v_2 + 1 \\ \max(v_3, v_2 + 1) \end{bmatrix}. \end{aligned}$$

- For $v_1 = 1$:

$$f(v) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix} \begin{bmatrix} 1 \\ 0 \\ v_3 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 + 1 \\ v_3 + 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ v_3 \end{bmatrix}$$

Since $v_3 \in \mathbb{Z}^+ \implies v_3 \geq 1$, we can substitute $v_3 = \max(v_3, 1)$, yielding $[1 \parallel 1 \parallel \max(v_3, 1)]^\top$.

Note that, the first component of $f(v)$ can be even assigned to 0, when $v_1 = 1$, by changing $W_2[0, 0] = 0$. $\square$

Theorem 13. A three-layer single-head Transformer decoder can find the width of a tree $\text{wd}(G_T)$ during the simulation of Dkst in exactly $n - 1$ steps, where n denotes the number of vertices in G_T .

Proof. To accommodate the computation of $\text{wd}(G_T)$, we augment the vectors $\tilde{x}_0, \dots, \tilde{x}_{n-1}, \tilde{y}_0, \dots, \tilde{y}_t$ as shown in Theorem 6 with three scalar coordinates such that

$$x_i = [\tilde{x}_i^\top \parallel 0 \parallel 0 \parallel 0]^\top \text{ and } y_0 = [\tilde{y}_0^\top \parallel 0 \parallel 1 \parallel 1]^\top$$

$\in \mathbb{R}^d$, where $d = 5n + 4$. The appended positions store, in order, the difference in distances $\pi_{\text{crd}}(\tilde{y}_t)$ and $\pi_{\text{crd}}(\tilde{y}_{t-1})$, maps $\text{twd}^{(t)}$ and $\text{mwd}^{(t)}$. We will use the selectors $\pi_{\text{dif}}, \pi_{\text{twd}}$, and $\pi_{\text{mwd}} : \mathbb{R}^d \rightarrow \mathbb{R}$ to select these positions along with reusing the previous selectors as in Theorem 6 as and when necessary.

The two layers of Theorem 6 remain identical except for $W_2^{(2)}$; the dimensional extension does not affect the previous computation once the parameters are adapted accordingly. In addition to the designated changes $W_2^{(2)}$ does in $y_t^{(1)'} (as in $\tilde{y}_t^{(1)'}$), it will set $\pi_{\text{dif}}(y_t^{(1)'})$ to $\sum_{i=0}^{n-1} \pi_{\text{buf}}(y_t^{(1)'})[i] - \pi_{\text{crd}}(y_t^{(1)'})$ to obtain $y_t^{(2)}$. Note that, $\pi_{\text{crd}}(y_t^{(1)'}) = \pi_{\text{crd}}(y_t)$.$

The third layer contains a trivial attention layer that takes $W_O^{(3)}$ as empty. Since the tree G_T is (un)weighted taking weights in $\{1, \infty\}$, and as we have already observed $\pi_{\text{dif}} \in \{0, 1\}$ in Observation 1, the subsequent FFN layer can be developed analogous to Lemma 12 considering $V_{\text{max}} = n$. Consider this as the resultant vector y_{t+1} . Clearly, $\pi_{\text{mwd}}(y_{n-1}) = \text{wd}(G_T)$. $\square$

Lemma 14. Let $v \in \mathbb{R}^{H+2}$ be a vector defined as the concatenation $v = [m \parallel h \parallel p_1 \parallel p_2 \parallel \dots \parallel p_H]^\top$, where $H > h \geq 1$ is a strictly bounding integer constant, $p_i \geq 1$ for all $i \in \{1, \dots, H\}$, and $m \in \{-1, 1\}$. There exists an FFN with ReLU activation, such that:

$$v' = \begin{cases} [m \parallel h + 1 \parallel p_1 \parallel \dots \parallel p_{h+1} + 1 \parallel \dots \parallel p_H]^\top & \text{if } m = 1 \\ [m \parallel h - 1 \parallel p_1 \parallel p_2 \parallel \dots \parallel p_H]^\top & \text{if } m = -1 \end{cases} \quad (9)$$

Proof. Let the input dimension be $d_{\text{in}} = H + 2$. We construct a hidden layer of dimension $d_h = 4H$ to evaluate the conditional mapping. We partition the hidden dimension into two functional blocks: state preservation mappings and discrete pulse functions. The weight matrices $W_1 \in \mathbb{R}^{4H \times (H+2)}$ and $W_2 \in \mathbb{R}^{(H+2) \times 4H}$ are constructed by concatenating their respective sub-matrices.

Since $m \in \{-1, 1\}$, we map it to the positive domain using two nodes: $\max(0, m)$ and $\max(0, -m)$. The state preservation sub-matrix $W_{1,\text{id}} \in \mathbb{R}^{(H+3) \times (H+2)}$ is defined as:

$$W_{1,\text{id}} = \begin{pmatrix} 1 & 0 & \mathbf{0}_{1 \times H} \\ -1 & 0 & \mathbf{0}_{1 \times H} \\ 0 & 1 & \mathbf{0}_{1 \times H} \\ \mathbf{0}_{H \times 1} & \mathbf{0}_{H \times 1} & I_H \end{pmatrix}$$

where I_H is the identity matrix. Since $h \geq 1$ and $p_i \geq 1$, the ReLU acts as the identity function for all these dimensions. To selectively increment p_{h+1} when $m = 1$, we construct an indicator for each $k \in \{1, \dots, H-1\}$ that evaluates to 1 when $m = 1 \wedge h = k$, and 0 otherwise. This requires 3 nodes per index k to form a piecewise linear basis. We stack $H-1$ blocks of size $3 \times (H+2)$ to form $W_{1,\text{pls}}$. The k -th block is:

$$W_{1,\text{pls}}^{(k)} = \begin{pmatrix} -(k-1) & 1 & \mathbf{0}_{1 \times H} \\ -k & 1 & \mathbf{0}_{1 \times H} \\ -(k+1) & 1 & \mathbf{0}_{1 \times H} \end{pmatrix}$$

The second layer matrix $W_2 = [W_{2,\text{id}} \parallel W_{2,\text{pls}}]$ linearly combines the hidden representations to yield v' .

First, $W_{2,\text{id}} \in \mathbb{R}^{(H+2) \times (H+3)}$ recovers the unmodified states and structurally applies the m increment to h :

$$W_{2,\text{id}} = \begin{pmatrix} 1 & -1 & 0 & \mathbf{0}_{1 \times H} \\ 1 & -1 & 1 & \mathbf{0}_{1 \times H} \\ \mathbf{0}_{H \times 1} & \mathbf{0}_{H \times 1} & \mathbf{0}_{H \times 1} & I_H \end{pmatrix}$$

Finally, $W_{2,\text{pls}} \in \mathbb{R}^{(H+2) \times 3(H-1)}$ routes the linear coefficients $[1, -2, 1]$ to the corresponding coordinates for p_2 through p_H (noting p_1 is never modified since $h \geq 1 \implies h+1 \geq 2$):

$$W_{2,\text{pls}} = \begin{pmatrix} \mathbf{0}_{3 \times 3} & \mathbf{0}_{3 \times 3} & \dots & \mathbf{0}_{3 \times 3} \\ [1 \parallel -2 \parallel 1] & \mathbf{0}_{1 \times 3} & \dots & \mathbf{0}_{1 \times 3} \\ \mathbf{0}_{1 \times 3} & [1 \parallel -2 \parallel 1] & \dots & \mathbf{0}_{1 \times 3} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0}_{1 \times 3} & \mathbf{0}_{1 \times 3} & \dots & [1 \parallel -2 \parallel 1] \end{pmatrix}$$

By construction, if $m = 1$, the k -th discrete pulse evaluates to $\max(0, h-k+1) - 2\max(0, h-k) + \max(0, h-k-1)$. Because h is integral, this yields exactly 1 at $h = k$ and 0 otherwise. Conversely, if $m = -1$, the arguments to the ReLUs become $h+k-1$, $h+k$, and $h+k+1$. Because $h \geq 1$ and $k \geq 1$, we are guaranteed that $h+k-1 \geq 1 > 0$. Consequently, all arguments are strictly positive, placing the ReLUs purely in their linear regime. The combination perfectly collapses to $(h+k-1) - 2(h+k) + (h+k+1) = 0$. Thus, no pointer p_i is affected when $m = -1$, satisfying the lemma. $\square$

Lemma 15. Let $v \in \mathbb{R}^{H+1}$ be a vector defined as the concatenation $v = [p_1 \parallel p_2 \parallel \dots \parallel p_H \parallel wd]^\top$, where $wd \geq 0$ and $p_i \geq 0$ for all $i \in \{1, \dots, H\}$. Assume that $p_i \leq wd + 1$ for all i , and that $p_i = wd + 1$ for at most one index i . There exists an FFN with ReLU activation, such that:

$$v' = \begin{cases} [p_1 \parallel p_2 \parallel \dots \parallel p_H \parallel wd + 1]^\top & \text{if } \exists i \text{ s.t. } p_i = wd + 1 \\ v & \text{otherwise} \end{cases} \quad (10)$$

Proof. Let the input dimension be $d_{\text{in}} = H+1$. We construct a hidden layer of dimension $d_h = 2H+1$. We partition the hidden layer into two sets of nodes: an identity mapping to preserve the input state, and a set of H temporary indicator nodes to detect the increment condition.

The first weight matrix $W_1 \in \mathbb{R}^{(2H+1) \times (H+1)}$ is constructed as a block matrix:

$$W_1 = \begin{pmatrix} I_{H+1} \\ I_H - \mathbf{1}_H \end{pmatrix}$$

where I_k denotes the $k \times k$ identity matrix, and $\mathbf{1}_H \in \mathbb{R}^{H \times 1}$ is the all-ones column vector.

Applying the ReLU activation yields the hidden state $h = \max(0, W_1 v) \in \mathbb{R}^{2H+1}$. Because $p_i \geq 0$ and $wd \geq 0$, the first $H+1$ dimensions pass through the ReLU unchanged, preserving the original vector v . The remaining H dimensions compute the temporary variables $tmp_i = \max(0, p_i - wd)$ for $i \in \{1, \dots, H\}$.

Given the constraint $p_i \leq wd + 1$, the difference $p_i - wd$ is bounded above by 1. Consequently, the temporary variables evaluate to:

$$tmp_i = \begin{cases} 1 & \text{if } p_i = wd + 1 \\ 0 & \text{if } p_i \leq wd \end{cases}$$

Since $p_i = wd + 1$ holds for at most one index, the vector $tmp = [tmp_1 \parallel \dots \parallel tmp_H]^\top$ is guaranteed to be either a one-hot vector or the zero vector.

The second weight matrix $W_2 \in \mathbb{R}^{(H+1) \times (2H+1)}$ reconstructs the output dimension and aggregates the temporary variables into the wd coordinate:

$$W_2 = \begin{pmatrix} I_H & \mathbf{0}_{H \times 1} & \mathbf{0}_{H \times H} \\ \mathbf{0}_{1 \times H} & 1 & \mathbf{1}_H^\top \end{pmatrix}$$

Multiplying h by W_2 leaves the components $p_1 \dots p_H$ strictly isolated and unmodified. The final coordinate is computed as $wd' = wd + \sum_{i=1}^H tmp_i$. Because tmp contains at most a single 1 corresponding to the condition $p_i = wd + 1$, the sum evaluates to 1 if the condition is met, and 0 otherwise. $\square$

Theorem 16. *A two-layer single-head Transformer decoder can find $\text{wd}(\psi(w))$ for a Dyck word w in exactly n CoT steps, where $|w| = n$.*

Proof. Given a Dyck word w of length n , $\text{ht}(\psi(w))$ is bounded above by $\frac{n}{2} + 1$, that is, when $\psi(w)$ is a path. The t^{th} decoding step records the width $\text{ht}(\psi(w))$ of the partial tree obtained after reading the prefix $w_0 \dots w_{t-1}$, equivalently the lattice path $\{(\tau, s_\tau)\}_{\tau=0}^t$. The embedding dimension is $d = \frac{3n}{2} + 4$, split into five blocks: the position $pos \in \mathbb{R}^n$ in one-hot form; the next position is a scalar holding the step $q_t \in \{-1, +1\}$ on an input token; the next is another scalar storing the cumulative sum of all q_τ see till the current CoT step. From the remaining block of length $\frac{n}{2} + 1$, p_i stores $|\{\tau \mid s_\tau = i \text{ and } q_\tau = s_\tau - s_{\tau-1} = +1\}|$ for $0 \leq \tau \leq t$ and $i \in \{1, \dots, \frac{n}{2}\}$. And the final block wd is a scalar that stores the maximum of all p_i , which turns out to be the width, as claimed in [Proposition 3](#).

Assume $\mathcal{E}_n = \{e_0, \dots, e_{n-1}\}$ denotes the standard basis vector on $\mathbb{R}^n$. The input tokens $x_0, \dots, x_{n-1} \in \mathbb{R}^d$ encodes the lattice path $\{\tau, s_\tau\}_{\tau=0}^n$ as follows:

$$x_\tau = [e_\tau \parallel q_\tau \parallel 0 \parallel 0^{\frac{n}{2}+1} \parallel 0]^\top.$$

The Dynamic is carried out by tokens $y_t \in \mathbb{R}^d$ with

$$y_0 = [e_0 \parallel 0 \parallel 0 \parallel 0^{\frac{n}{2}+1} \parallel 0]^\top.$$

The selectors $\pi_{pos} : \mathbb{R}^d \rightarrow \mathbb{R}^n$ selects the first one-hot block that corresponds to the current position, $\pi_q : \mathbb{R}^d \rightarrow \{-1, 0, 1\}$ selects the block that records the corresponding move taken at the position for input tokens or 0 for generated tokens, $\pi_{ht} : \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$ records the height, $\pi_{p_i} : \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$ records p_i as discussed above. Finally, $\pi_{pos} : \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$ records the wd . Analogous to [Theorem 11](#), the attention mechanism of the first layer employs a shift matrix S to find the π_{pos} and records the corresponding π_q from the input tokens. Let α' be the vector after multiplication by $W_O^{(1)}$ where every block is 0 except $\pi_q(\alpha') = q_\tau$. After the residual connection with y_t , let this vector be $\tilde{y}_t$. We employ $W_1^{(1)}$ and $W_2^{(1)}$ of the following FFN layer similar to [Lemma 14](#) so that $[\pi_q(\tilde{y}_t) \parallel \pi_{ht}(\tilde{y}_t) \parallel \pi_{p_1}(\tilde{y}_t) \parallel \dots \parallel \pi_{\frac{n}{2}+1}(\tilde{y}_t)]^\top$ serves the vector v and undergoes an update as in (9). The remaining positions remain unchanged.

Note that the above operation would impact only one position of all $p_1, \dots, p_{\frac{n}{2}+1}$. For the second layer, we will utilize the FFN only. We will consider $w_O^{(1)}$ as $\mathbf{0}$. Let the vector obtained after applying the residual connection be $\tilde{y}_t^{(1)}$. The FFN layer performs updates on $[\pi_{p_1}(\tilde{y}_t^{(1)}) \parallel \dots \parallel \pi_{\frac{n}{2}+1}(\tilde{y}_t^{(1)}) \parallel wd]^\top$ similar to (10) as shown in [Lemma 15](#). The linear projections do not affect any other positions of $\tilde{y}_t^{(1)}$ except that $W_2^{(2)}$ updates the the block q to 0. The resultant vector is y_{t+1} and $\pi_{wd}(y_n) = \text{wd}(\psi(w))$. $\square$